V2X Message Classification with Attention

Full Working Source Code

GitHub repository: https://github.com/Suhani1589/v2x-message-classification

This document contains the complete current source code required to run the proposed Streamlit research dashboard and the underlying ML pipeline (dataset preparation, model definitions, training, evaluation, and inference). Code is copied verbatim from the project files; paths and filenames are preserved.

Contents (included files)

1. `requirements.txt` — 11 lines
2. `config.yaml` — 78 lines
3. `frontend/__init__.py` — 1 lines
4. `frontend/app.py` — 438 lines
5. `frontend/charts.py` — 118 lines
6. `frontend/inference_bridge.py` — 46 lines
7. `frontend/metrics_data.py` — 113 lines
8. `frontend/repo_info.py` — 30 lines
9. `frontend/styles.py` — 300 lines
10. `v2x_cls/__init__.py` — 1 lines
11. `v2x_cls/config.py` — 24 lines
12. `v2x_cls/seeds.py` — 26 lines
13. `v2x_cls/data.py` — 725 lines
14. `v2x_cls/preprocessing.py` — 135 lines
15. `v2x_cls/training.py` — 77 lines
16. `v2x_cls/evaluation.py` — 99 lines
17. `v2x_cls/visualization.py` — 108 lines
18. `v2x_cls/models/__init__.py` — 174 lines
19. `v2x_cls/models/attention.py` — 75 lines
20. `models/__init__.py` — 20 lines
21. `scripts/prepare_dataset.py` — 102 lines
22. `scripts/train.py` — 110 lines
23. `scripts/evaluate.py` — 135 lines
24. `scripts/predict.py` — 147 lines
25. `scripts/run_experiment.py` — 98 lines

Excluded from this PDF (not required as source for submission / too large / non-source): `.venv/`, `.git/`, `__pycache__/`, raw/processed datasets, model checkpoints (`*.keras`), experiment plots/metrics artifacts, notebooks, OS junk (`.DS_Store`), and temporary caches.

To run: install dependencies from `requirements.txt`, use `config.yaml`, prepare data / train via `scripts/`, then `streamlit run frontend/app.py`. Trained weights are available in the GitHub repository under `checkpoints/`.

1. requirements.txt

Exact path: `requirements.txt`

```
tensorflow>=2.13,<2.16
numpy>=1.23
pandas>=1.5
scikit-learn>=1.2
matplotlib>=3.7
seaborn>=0.12
PyYAML>=6.0
tabulate>=0.9
tqdm>=4.65
streamlit>=1.28
plotly>=5.18
```

2. config.yaml

Exact path: `config.yaml`

```
# Experiment configuration for V2X message classification.
# Values marked "paper" come from Lazaga et al. (ICoAIT 2025).
# Values marked "assumption" are not stated in the paper and are documented choices.

project:
  name: v2x-message-classification
  seed: 42

paths:
  raw_data_dir: data/raw
  processed_data_dir: data/processed
  checkpoints_dir: checkpoints
  results_dir: experiments/results
  # Preferred official filename from IEEE DataPort:
  official_csv_candidates:
    - "V2X_train - Sheet1.csv"
    - "V2X_train-Sheet1.csv"
    - "v2x_messages.csv"
  # Used only when the official CSV is unavailable (see README / PAPER_NOTES).
  synthetic_csv: data/raw/v2x_messages_synthetic.csv

data:
  text_column: Message Content
  label_column: Message Type
  # Paper label mapping (Section II.D):
  label_order:
    - Emergency
    - Hazard
    - Roadwork
    - Spam
    - Traffic
    - Weather
  # Paper used 938 of ~1000 IEEE DataPort rows; exact filter not disclosed.
  # Assumption: use all valid rows with Message Content + Message Type present.
  train_ratio: 0.70
  val_ratio: 0.15
  test_ratio: 0.15
  # Paper reports validation_split=0.3 with no held-out test set.
  # We keep a fixed test set for scientific model comparison (assumption).

preprocessing:
  max_sequence_length: 12   # paper
  padding: post             # paper
  truncating: post          # assumption (paired with post-padding)
  lowercase: true           # Keras Tokenizer default behavior
  oov_token: "<OOV>"        # assumption (improves robustness; not in paper)
  # vocab_size: null means use all tokens seen in the training split

model:
  # Architecture sizes are NOT specified in the paper ("Keras default parameters").
  # Assumptions below are conservative and kept identical across all models.
  embedding_dim: 64
  rnn_units: 64
  cnn_filters: 64
  cnn_kernel_size: 3
  dense_units: 64
  # Paper: first Dense uses sigmoid, second Dense uses softmax.
  use_paper_dense_stack: true
  dropout: 0.0              # paper does not mention dropout

training:
  epochs: 10                # paper
  batch_size: 32            # assumption
  learning_rate: 0.001      # assumption (Adam default)
  optimizer: adam
  # Paper loss: CategoricalCrossentropy (Section II.E)
  loss: categorical_crossentropy
  early_stopping_patience: 5  # extra safeguard (assumption); set null to disable
  checkpoint_monitor: val_loss
  checkpoint_mode: min

evaluation:
  average_modes:
    - macro
    - weighted

inference:
  top_k_attention_tokens: 5
```

3. frontend/__init__.py

Exact path: `frontend/__init__.py`

```
# Frontend package marker (Streamlit entry is app.py).
```

4. frontend/app.py

Exact path: `frontend/app.py`

```
"""Streamlit research dashboard for V2X CNN-BRNN + Attention.

Run from the repository root:
  streamlit run frontend/app.py

This UI is intentionally separate from training/evaluation code. Inference
reuses scripts/predict.py and existing checkpoints when available.
"""

from __future__ import annotations

import sys
import warnings
from pathlib import Path

import pandas as pd
import streamlit as st

# Plotly emits a noisy deprecation via Streamlit's chart wrapper on some versions.
warnings.filterwarnings(
    "ignore",
    message=".*keyword arguments have been deprecated.*",
)

ROOT = Path(__file__).resolve().parents[1]
FRONTEND = Path(__file__).resolve().parent
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
if str(FRONTEND) not in sys.path:
    sys.path.insert(0, str(FRONTEND))

from charts import (  # noqa: E402
    accuracy_chart,
    conceptual_attention_figure,
    f1_chart,
    loss_chart,
    real_attention_figure,
)
from inference_bridge import try_prediction  # noqa: E402
from metrics_data import (  # noqa: E402
    BEST_ACCURACY_PCT,
    BEST_F1_PCT,
    BEST_MODEL_NAME,
    CONCLUSION,
    IMPROVEMENTS,
    MODELS,
    RESEARCH_QUESTION,
    SYNTHETIC_LIMITATION,
    model_by_key,
)
from repo_info import artifacts_ready, checkpoint_status, load_dataset_report  # noqa: E402
from styles import APP_CSS, architecture_html  # noqa: E402

st.set_page_config(
    page_title="CNN-BRNN + Attention | Research Dashboard",
    page_icon="■",
    layout="wide",
    initial_sidebar_state="collapsed",
)

st.markdown(f"<style>{APP_CSS}</style>", unsafe_allow_html=True)


def section(title: str) -> None:
    st.markdown(f"## {title}")


def show_chart(fig) -> None:
    st.plotly_chart(fig, use_container_width=True, config={"displayModeBar": False})


# ---------------------------------------------------------------------------
# 1. Hero
# ---------------------------------------------------------------------------
st.markdown(
    f"""
<div class="hero">
  <div class="badge-row">
    <span class="badge">Research Project</span>
    <span class="badge">Synthetic Dataset</span>
    <span class="badge">Sequence Classification</span>
  </div>
  <h1>Sequence Classification with CNN-BRNN and Attention</h1>
  <div class="subtitle">
    Comparative Analysis of BRNN, CNN-BRNN, and CNN-BRNN with Attention
  </div>
```

```
  <p style="margin:0;max-width:48rem;color:rgba(248,250,252,0.9);line-height:1.55;">
    This dashboard presents a controlled comparison of three deep sequence models
    for multiclass V2X message classification. The study asks whether adding
    convolutional feature extraction and attention improves over a standalone
    bidirectional RNN — under a clearly labeled synthetic experimental setting.
  </p>
  <div class="metric-pills">
    <div class="metric-pill">
      <div class="label">Best observed Accuracy</div>
      <div class="value">{BEST_ACCURACY_PCT}</div>
      <div style="font-size:0.82rem;opacity:0.8;">{BEST_MODEL_NAME}</div>
    </div>
    <div class="metric-pill">
      <div class="label">Best observed Macro F1</div>
      <div class="value">{BEST_F1_PCT}</div>
      <div style="font-size:0.82rem;opacity:0.8;">{BEST_MODEL_NAME}</div>
    </div>
  </div>
</div>
""",
    unsafe_allow_html=True,
)

# ---------------------------------------------------------------------------
# 2. Research question
# ---------------------------------------------------------------------------
section("Research Question")
st.markdown(
    f"""
<div class="section-card rq-card">
  <div style="font-size:0.78rem;font-weight:700;letter-spacing:0.05em;text-transform:uppercase;color:#0F766E;">
    Central question
  </div>
  <p class="rq-text">“{RESEARCH_QUESTION}”</p>
  <div style="font-size:0.9rem;color:#334155;margin-bottom:0.35rem;"><b>Experimental progression</b></div>
  <div class="flow">
    <span class="flow-step">BRNN</span><span class="flow-arrow">→</span>
    <span class="flow-step">Bidirectional temporal modeling</span>
  </div>
  <div class="flow">
    <span class="flow-step">CNN + BRNN</span><span class="flow-arrow">→</span>
    <span class="flow-step">Spatial/local feature extraction + bidirectional temporal modeling</span>
  </div>
  <div class="flow">
    <span class="flow-step">CNN + BRNN + Attention</span><span class="flow-arrow">→</span>
    <span class="flow-step">Spatial features + bidirectional temporal modeling + focus on informative parts of the sequence</span>
  </div>
</div>
""",
    unsafe_allow_html=True,
)

# ---------------------------------------------------------------------------
# 3. Architectures
# ---------------------------------------------------------------------------
section("Model Architectures")
st.markdown(architecture_html(), unsafe_allow_html=True)
with st.expander("Beginner-friendly component guide", expanded=False):
    st.markdown(
        """
- **Input sequence** — tokenized message text padded/truncated to a fixed length.
- **CNN** — scans short windows of tokens to detect local patterns.
- **BRNN** — processes the sequence in both directions for surrounding context.
- **Flatten** — concatenates all timestep vectors (used by BRNN / CNN-BRNN).
- **Attention** — learns a weight for each timestep and forms a weighted summary.
- **Classification head** — maps the summary vector to one of six message classes.
"""
    )

# ---------------------------------------------------------------------------
# 4. Dataset
# ---------------------------------------------------------------------------
section("Dataset")
report = load_dataset_report()
st.markdown(
    f"""
<div class="section-card">
  <div class="badge-row" style="margin-bottom:0.75rem;">
    <span class="badge" style="background:#0F766E;border:none;">Dataset Type: Synthetic</span>
  </div>
  <p style="color:#334155;line-height:1.55;margin-top:0;">
    Experiments were run on a <b>synthetic</b> V2X-style message corpus constructed
    for controlled model comparison. The official IEEE DataPort CSV is not required
    for this dashboard; the synthetic setting remains intentional and is stated
    throughout the results.
  </p>
  <div class="limit-box">{SYNTHETIC_LIMITATION}</div>
</div>
""",
    unsafe_allow_html=True,
)
```

```
if report:
    cols = st.columns(4)
    cols[0].metric("Rows", f"{report.get('n_rows', '—')}")
    cols[1].metric("Vocab size", f"{report.get('vocab_size', '—')}")
    cols[2].metric("Max sequence length", f"{report.get('max_sequence_length', '—')}")
    cols[3].metric("Formulation", str(report.get("formulation", "—")))
    if report.get("class_counts"):
        st.caption("Class counts from `data/processed/dataset_report.json` (current prepared split source).")
        st.dataframe(
            pd.DataFrame(
                [{"Class": k, "Count": v} for k, v in report["class_counts"].items()]
            ),
            hide_index=True,
            width="stretch",
        )
    src = report.get("source")
    if src:
        st.caption(f"Dataset report source key: `{src}`")
else:
    st.info("Dataset report not found at `data/processed/dataset_report.json`. Metrics below still use the reported experimental figures.")

# ---------------------------------------------------------------------------
# 5. Performance dashboard
# ---------------------------------------------------------------------------
section("Model Performance Dashboard")
st.caption("Reported experimental results on the synthetic dataset (display figures).")

c1, c2, c3 = st.columns(3)
with c1:
    show_chart(accuracy_chart())
with c2:
    show_chart(f1_chart())
with c3:
    show_chart(loss_chart())

table = pd.DataFrame(
    [
        {
            "Model": m["name"],
            "Accuracy": m["accuracy_pct"],
            "Macro F1": m["f1_pct"],
            "Loss": f"{m['loss']:.3f}",
        }
        for m in MODELS
    ]
)
st.dataframe(table, hide_index=True, width="stretch")
st.markdown(
    """
- Highest **Accuracy** and **Macro F1**: <span class="highlight">CNN-BRNN + Attention</span>
- Lowest **Loss**: <span class="highlight">CNN-BRNN (0.274)</span> — Attention loss is **0.275** (nearly identical; not lower)
""",
    unsafe_allow_html=True,
)

# ---------------------------------------------------------------------------
# 6. Improvements
# ---------------------------------------------------------------------------
section("Performance Improvement")
st.markdown('<div class="delta-grid">', unsafe_allow_html=True)
left, right = st.columns(2)
for col, key in ((left, "brnn_to_cnn"), (right, "cnn_to_attn")):
    block = IMPROVEMENTS[key]
    with col:
        st.markdown(
            f"""
<div class="delta-card">
  <h4>{block["title"]}</h4>
  <div class="delta-row"><span>Accuracy</span><span><span class="delta-from">{block["accuracy"][0]}</span> → <span class="delta-to">{block["ac
  <div class="delta-row"><span>Macro F1</span><span><span class="delta-from">{block["f1"][0]}</span> → <span class="delta-to">{block["f1"][1]}
</div>
""",
            unsafe_allow_html=True,
        )

# ---------------------------------------------------------------------------
# 7. Attention explanation
# ---------------------------------------------------------------------------
section("Attention Mechanism")
st.markdown(
    """
<div class="section-card">
  <p style="color:#334155;line-height:1.55;">
    Attention allows the model to assign different importance to different parts
    of the sequence, helping it focus on the most informative representations.
  </p>
  <div class="flow">
    <span class="flow-step">t1</span><span class="flow-arrow">→</span>
    <span class="flow-step">t2</span><span class="flow-arrow">→</span>
    <span class="flow-step">t3</span><span class="flow-arrow">→</span>
    <span class="flow-step">t4</span><span class="flow-arrow">→</span>
```

```
    <span class="flow-step">t5</span>
  </div>
  <div class="flow">
    <span class="flow-step">low</span><span class="flow-arrow">→</span>
    <span class="flow-step">low</span><span class="flow-arrow">→</span>
    <span class="flow-step">high</span><span class="flow-arrow">→</span>
    <span class="flow-step">high</span><span class="flow-arrow">→</span>
    <span class="flow-step">low</span>
  </div>
</div>
""",
    unsafe_allow_html=True,
)
st.plotly_chart(conceptual_attention_figure(), use_container_width=True, config={"displayModeBar": False})
st.caption("Conceptual illustration — not measured weights from a specific prediction.")

# ---------------------------------------------------------------------------
# 8. Interactive model comparison
# ---------------------------------------------------------------------------
section("Interactive Model Comparison")
choice = st.selectbox(
    "Select a model",
    options=[m["key"] for m in MODELS],
    format_func=lambda k: model_by_key(k)["name"],
)
selected = model_by_key(choice)
m1, m2, m3 = st.columns(3)
m1.metric("Accuracy", selected["accuracy_pct"])
m2.metric("Macro F1", selected["f1_pct"])
m3.metric("Loss", f"{selected['loss']:.3f}")
st.markdown("**Architecture**")
st.code(" → ".join(selected["pipeline"]))
st.write(selected["blurb"])

# ---------------------------------------------------------------------------
# 9. Inference demo
# ---------------------------------------------------------------------------
section("Inference / Demo")
ckpts = checkpoint_status()
ready = artifacts_ready() and any(ckpts.values())

if not ready:
    st.warning(
        "Trained-model inference is available in this repository via "
        "`scripts/predict.py`, but checkpoints and/or preprocessing artifacts "
        "were not found in the expected locations. No fake predictions are shown."
    )
    st.markdown(
        """
**Model Demo (offline)**
Once `checkpoints/*/best.keras` and `data/processed/artifacts/` are present, this
section loads the real models. Until then, use the CLI:

```bash
python scripts/predict.py --model cnn_brnn_attention --text "Emergency vehicle approaching intersection"
```
"""
)
else:
    st.success("Checkpoints and tokenizer artifacts detected — live inference enabled.")
    missing = [k for k, ok in ckpts.items() if not ok]
    if missing:
        st.info("Missing checkpoints for: " + ", ".join(missing))

    demo_options = [k for k, ok in ckpts.items() if ok]
    default_infer = (
        "cnn_brnn_attention"
        if "cnn_brnn_attention" in demo_options
        else demo_options[0]
)
    demo_model = st.selectbox(
        "Inference model",
        options=demo_options,
        index=demo_options.index(default_infer),
        format_func=lambda k: model_by_key(k)["name"],
        key="infer_model",
)
    text = st.text_area(
        "V2X message text",
        value="Emergency vehicle approaching intersection",
        height=90,
)
    if st.button("Run prediction", type="primary"):
        with st.spinner("Running model…"):
            result, err = try_prediction(text, demo_model)
        if err:
            st.error(err)
        elif result:
            st.markdown(f"**Predicted class:** `{result['predicted_class']}`")
            st.markdown(f"**Confidence:** {result['confidence_percent']}%")
            prob_df = pd.DataFrame(
```

```
                [
                    {"Class": k, "Probability": v}
                    for k, v in result["class_probabilities"].items()
                ]
            ).sort_values("Probability", ascending=False)
            st.dataframe(prob_df, hide_index=True, width="stretch")
            if "attention_weights" in result:
                pairs = [p for p in result["attention_weights"] if p["token"] != "<PAD>"]
                if pairs:
                    st.markdown("**Real attention weights** from the companion attention model")
                    st.caption(result.get("attention_note", ""))
                    show_chart(
                        real_attention_figure(
                            [p["token"] for p in pairs],
                            [p["weight"] for p in pairs],
                            f"Attention weights — {result['predicted_class']}",
                        )
                    )
                    if result.get("important_tokens"):
                        st.write("Top tokens:", ", ".join(result["important_tokens"]))

# ---------------------------------------------------------------------------
# 10. Conclusion
# ---------------------------------------------------------------------------
section("Research Conclusion")
st.markdown(
    f"""
<div class="section-card rq-card">
  <p class="rq-text" style="font-size:1.02rem;">{CONCLUSION}</p>
</div>
""",
    unsafe_allow_html=True,
)

# ---------------------------------------------------------------------------
# 11. Limitations
# ---------------------------------------------------------------------------
section("Limitations")
st.markdown(
    f"""
<div class="section-card">
  <ul style="color:#334155;line-height:1.6;">
    <li><b>Synthetic dataset</b> — {SYNTHETIC_LIMITATION}</li>
    <li>Results may not directly generalize to real-world V2X traffic.</li>
    <li>Further validation on real-world datasets would be required before operational claims.</li>
    <li>Additional experiments could evaluate robustness and generalization more thoroughly.</li>
  </ul>
</div>
""",
    unsafe_allow_html=True,
)

# ---------------------------------------------------------------------------
# 12. Future work
# ---------------------------------------------------------------------------
section("Future Work")
st.markdown(
    """
<div class="section-card">
  <ul style="color:#334155;line-height:1.6;">
    <li>Evaluation on real-world datasets</li>
    <li>Larger and more diverse datasets</li>
    <li>Cross-dataset validation</li>
    <li>Hyperparameter tuning</li>
    <li>More extensive ablation studies</li>
    <li>Broader attention visualization studies using real model weights</li>
  </ul>
  <p class="footnote">These items are proposed next steps — they have not been completed in this dashboard.</p>
</div>
""",
    unsafe_allow_html=True,
)

st.markdown("---")
st.caption(
    "UI module: `frontend/` · Research code unchanged under `v2x_cls/` and `scripts/` · "
    "Synthetic-dataset experiment demo"
)
```

5. frontend/charts.py

Exact path: `frontend/charts.py`

```
"""Plotly chart builders for the research dashboard."""

from __future__ import annotations

from typing import List

import plotly.graph_objects as go

from metrics_data import MODELS

# Academic palette: ink / slate / teal — no purple-forward theme.
_COLORS = {
    "brnn": "#64748B",
    "cnn_brnn": "#0F766E",
    "cnn_brnn_attention": "#0B3A53",
}


def _base_layout(title: str, y_title: str, y_range=None) -> dict:
    layout = dict(
        title=dict(text=title, font=dict(size=16, color="#0F172A", family="IBM Plex Sans")),
        paper_bgcolor="rgba(0,0,0,0)",
        plot_bgcolor="rgba(248,250,252,0.9)",
        font=dict(family="IBM Plex Sans", color="#334155", size=13),
        margin=dict(l=48, r=24, t=56, b=48),
        yaxis=dict(title=y_title, gridcolor="#E2E8F0", zeroline=False),
        xaxis=dict(title="", gridcolor="#E2E8F0"),
        showlegend=False,
        height=340,
    )
    if y_range is not None:
        layout["yaxis"]["range"] = y_range
    return layout


def accuracy_chart() -> go.Figure:
    fig = go.Figure(
        go.Bar(
            x=[m["name"] for m in MODELS],
            y=[m["accuracy"] * 100 for m in MODELS],
            marker_color=[_COLORS[m["key"]] for m in MODELS],
            text=[m["accuracy_pct"] for m in MODELS],
            textposition="outside",
            hovertemplate="%{x}<br>Accuracy: %{y:.1f}%<extra></extra>",
        )
    )
    fig.update_layout(**_base_layout("Accuracy (%)", "Accuracy (%)", y_range=[85, 100]))
    return fig


def f1_chart() -> go.Figure:
    fig = go.Figure(
        go.Bar(
            x=[m["name"] for m in MODELS],
            y=[m["f1_macro"] * 100 for m in MODELS],
            marker_color=[_COLORS[m["key"]] for m in MODELS],
            text=[m["f1_pct"] for m in MODELS],
            textposition="outside",
            hovertemplate="%{x}<br>Macro F1: %{y:.1f}%<extra></extra>",
        )
    )
    fig.update_layout(**_base_layout("Macro F1 (%)", "Macro F1 (%)", y_range=[85, 100]))
    return fig


def loss_chart() -> go.Figure:
    # Do not specially highlight Attention as "best" on loss — CNN-BRNN is slightly lower.
    colors = [_COLORS[m["key"]] for m in MODELS]
    fig = go.Figure(
        go.Bar(
            x=[m["name"] for m in MODELS],
            y=[m["loss"] for m in MODELS],
            marker_color=colors,
            text=[f"{m['loss']:.3f}" for m in MODELS],
            textposition="outside",
            hovertemplate="%{x}<br>Loss: %{y:.3f}<extra></extra>",
        )
    )
    fig.update_layout(**_base_layout("Loss (lower is better)", "Loss", y_range=[0, 0.45]))
    return fig


def conceptual_attention_figure() -> go.Figure:
    """Clearly labeled conceptual illustration — not real model weights."""
    tokens = ["t1", "t2", "t3", "t4", "t5"]
    weights = [0.08, 0.10, 0.34, 0.32, 0.16]
```

```
    fig = go.Figure(
        go.Bar(
            x=tokens,
            y=weights,
            marker_color=["#94A3B8", "#94A3B8", "#0F766E", "#0F766E", "#94A3B8"],
            text=["low", "low", "high", "high", "low"],
            textposition="outside",
        )
    )
    fig.update_layout(
        **_base_layout(
            "Conceptual illustration — example attention weight pattern",
            "Relative weight",
            y_range=[0, 0.5],
        )
    )
    return fig


def real_attention_figure(tokens: List[str], weights: List[float], title: str) -> go.Figure:
    fig = go.Figure(
        go.Bar(
            x=tokens,
            y=weights,
            marker_color="#0B3A53",
            text=[f"{w:.2f}" for w in weights],
            textposition="outside",
            hovertemplate="Token: %{x}<br>Weight: %{y:.4f}<extra></extra>",
        )
    )
    fig.update_layout(**_base_layout(title, "Attention weight"))
    return fig
```

6. frontend/inference_bridge.py

Exact path: `frontend/inference_bridge.py`

```
"""Thin bridge from the Streamlit UI to existing CLI inference helpers.

Does not reimplement models — reuses scripts/predict.py via file load so we
do not need to alter the scripts/ package layout.
"""

from __future__ import annotations

import importlib.util
import sys
from pathlib import Path
from typing import Any, Dict, Optional, Tuple

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from v2x_cls.config import load_config


def _load_predict_module():
    path = ROOT / "scripts" / "predict.py"
    spec = importlib.util.spec_from_file_location("v2x_cls_predict_cli", path)
    if spec is None or spec.loader is None:
        raise ImportError(f"Cannot load prediction module from {path}")
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module


def run_prediction(text: str, model_name: str) -> Dict[str, Any]:
    """Call the existing predict_text pipeline."""
    predict_mod = _load_predict_module()
    cfg = load_config(str(ROOT / "config.yaml"))
    return predict_mod.predict_text(text.strip(), cfg, model_name, save_plot=None)


def try_prediction(text: str, model_name: str) -> Tuple[Optional[Dict[str, Any]], Optional[str]]:
    if not text.strip():
        return None, "Enter a V2X message to classify."
    try:
        return run_prediction(text, model_name), None
    except FileNotFoundError as exc:
        return None, f"Checkpoint or artifacts missing: {exc}"
    except Exception as exc:  # noqa: BLE001 — surface to UI gracefully
        return None, f"Inference failed: {exc}"
```

7. frontend/metrics_data.py

Exact path: `frontend/metrics_data.py`

```
"""Reported experimental metrics for the research dashboard.

These values match the completed synthetic-dataset experiment highlighted in the
UI brief (rounded display figures). They are intentionally fixed for the demo
so the dashboard does not drift if comparison CSVs are regenerated.
"""

from __future__ import annotations

from typing import Any, Dict, List

# Display metrics requested for the research dashboard.
MODELS: List[Dict[str, Any]] = [
    {
        "key": "brnn",
        "name": "BRNN",
        "full_name": "Bidirectional RNN",
        "accuracy": 0.915,
        "accuracy_pct": "91.5%",
        "f1_macro": 0.905,
        "f1_pct": "90.5%",
        "loss": 0.360,
        "pipeline": ["Input Sequence", "BRNN", "Classification"],
        "blurb": (
            "A bidirectional recurrent network reads the token sequence forward "
            "and backward, then flattens the timestep outputs for classification."
        ),
    },
    {
        "key": "cnn_brnn",
        "name": "CNN-BRNN",
        "full_name": "CNN + Bidirectional RNN",
        "accuracy": 0.943,
        "accuracy_pct": "94.3%",
        "f1_macro": 0.940,
        "f1_pct": "94.0%",
        "loss": 0.274,
        "pipeline": ["Input Sequence", "CNN Feature Extraction", "BRNN", "Classification"],
        "blurb": (
            "A convolutional front-end extracts local n-gram patterns before the "
            "bidirectional RNN models temporal context across the sequence."
        ),
    },
    {
        "key": "cnn_brnn_attention",
        "name": "CNN-BRNN + Attention",
        "full_name": "CNN + BRNN + Additive Attention",
        "accuracy": 0.950,
        "accuracy_pct": "95.0%",
        "f1_macro": 0.947,
        "f1_pct": "94.7%",
        "loss": 0.275,
        "pipeline": [
            "Input Sequence",
            "CNN Feature Extraction",
            "BRNN",
            "Attention",
            "Classification",
        ],
        "blurb": (
            "After CNN and bidirectional RNN encoding, additive attention pools "
            "timesteps by learned importance instead of flattening them uniformly."
        ),
    },
]

BEST_ACCURACY_PCT = "95.0%"
BEST_F1_PCT = "94.7%"
BEST_MODEL_NAME = "CNN-BRNN + Attention"

RESEARCH_QUESTION = (
    "How does the integration of CNN-based feature extraction and an attention "
    "mechanism with a Bidirectional Recurrent Neural Network (BRNN) affect "
    "sequence classification performance compared with a standalone BRNN and a "
    "CNN-BRNN model?"
)

CONCLUSION = (
    "Across the evaluated models, CNN-BRNN with Attention achieved the highest "
    "observed accuracy (95.0%) and Macro F1 score (94.7%). CNN-based feature "
    "extraction improved performance over the standalone BRNN, while adding "
    "attention produced a further improvement in accuracy and Macro F1. The "
    "loss values of CNN-BRNN (0.274) and CNN-BRNN with Attention (0.275) were "
    "nearly identical, so the improvement should be interpreted primarily "
    "through the classification metrics rather than loss reduction."
)
```

```
SYNTHETIC_LIMITATION = (
    "The experiments were conducted on a synthetic dataset; therefore, the "
    "reported performance demonstrates model behavior under the constructed "
    "experimental setting and does not by itself establish performance on "
    "real-world data."
)

IMPROVEMENTS = {
    "brnn_to_cnn": {
        "title": "BRNN → CNN-BRNN",
        "accuracy": ("91.5%", "94.3%"),
        "f1": ("90.5%", "94.0%"),
    },
    "cnn_to_attn": {
        "title": "CNN-BRNN → CNN-BRNN + Attention",
        "accuracy": ("94.3%", "95.0%"),
        "f1": ("94.0%", "94.7%"),
    },
}


def model_by_key(key: str) -> Dict[str, Any]:
    for m in MODELS:
        if m["key"] == key:
            return m
    raise KeyError(key)
```

8. frontend/repo_info.py

Exact path: `frontend/repo_info.py`

```
"""Helpers to load repository artifacts for the dashboard (read-only)."""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any, Dict, Optional

ROOT = Path(__file__).resolve().parents[1]


def load_dataset_report() -> Optional[Dict[str, Any]]:
    path = ROOT / "data" / "processed" / "dataset_report.json"
    if not path.exists():
        return None
    with path.open(encoding="utf-8") as f:
        return json.load(f)


def checkpoint_status() -> Dict[str, bool]:
    status = {}
    for name in ("brnn", "cnn_brnn", "cnn_brnn_attention"):
        ckpt = ROOT / "checkpoints" / name
        status[name] = (ckpt / "best.keras").exists() or (ckpt / "final.keras").exists()
    return status


def artifacts_ready() -> bool:
    arts = ROOT / "data" / "processed" / "artifacts"
    return (arts / "tokenizer.pkl").exists() and (arts / "label_mapping.json").exists()
```

9. frontend/styles.py

Exact path: `frontend/styles.py`

```
"""Shared CSS / HTML snippets for a research-lab aesthetic."""

from __future__ import annotations

APP_CSS = """
@import url('https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600;700&family=IBM+Plex+Serif:wght@500;600&display=swap');

:root {
  --ink: #0B1220;
  --slate: #334155;
  --muted: #64748B;
  --line: #E2E8F0;
  --panel: #FFFFFF;
  --soft: #F8FAFC;
  --teal: #0F766E;
  --navy: #0B3A53;
  --amber: #B45309;
}

html, body, [class*="css"] {
  font-family: "IBM Plex Sans", sans-serif;
  color: var(--ink);
}

.block-container {
  padding-top: 1.5rem;
  padding-bottom: 3rem;
  max-width: 1120px;
}

h1, h2, h3 {
  font-family: "IBM Plex Serif", Georgia, serif !important;
  letter-spacing: -0.02em;
  color: var(--ink) !important;
}

.hero {
  background: linear-gradient(160deg, #0B3A53 0%, #0F766E 100%);
  color: #F8FAFC;
  border-radius: 18px;
  padding: 2.4rem 2.2rem 2.2rem;
  margin-bottom: 1.5rem;
  box-shadow: 0 18px 40px rgba(11, 58, 83, 0.18);
}

.hero h1 {
  color: #FFFFFF !important;
  font-size: 2.15rem;
  margin: 0.35rem 0 0.55rem 0;
  line-height: 1.2;
}

.hero .subtitle {
  color: rgba(248, 250, 252, 0.88);
  font-size: 1.05rem;
  margin-bottom: 1.1rem;
  max-width: 46rem;
}

.badge-row {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
  margin-bottom: 0.9rem;
}

.badge {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  padding: 0.28rem 0.7rem;
  border-radius: 999px;
  font-size: 0.78rem;
  font-weight: 600;
  letter-spacing: 0.02em;
  background: rgba(255,255,255,0.14);
  border: 1px solid rgba(255,255,255,0.22);
  color: #fff;
}

.metric-pills {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 0.75rem;
  margin-top: 1.1rem;
}
```

```
.metric-pill {
  background: rgba(255,255,255,0.1);
  border: 1px solid rgba(255,255,255,0.18);
  border-radius: 12px;
  padding: 0.85rem 1rem;
}

.metric-pill .label {
  font-size: 0.78rem;
  color: rgba(248,250,252,0.75);
  text-transform: uppercase;
  letter-spacing: 0.04em;
}

.metric-pill .value {
  font-size: 1.55rem;
  font-weight: 700;
  margin-top: 0.15rem;
}

.section-card {
  background: var(--panel);
  border: 1px solid var(--line);
  border-radius: 16px;
  padding: 1.35rem 1.4rem;
  margin-bottom: 1rem;
  box-shadow: 0 8px 24px rgba(15, 23, 42, 0.04);
}

.rq-card {
  border-left: 4px solid var(--teal);
  background: #F0FDFA;
}

.rq-text {
  font-family: "IBM Plex Serif", Georgia, serif;
  font-size: 1.12rem;
  line-height: 1.55;
  color: var(--ink);
  margin: 0.4rem 0 0.8rem 0;
}

.flow {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 0.45rem;
  margin: 0.65rem 0;
}

.flow-step {
  background: #F1F5F9;
  border: 1px solid var(--line);
  color: var(--slate);
  padding: 0.4rem 0.7rem;
  border-radius: 8px;
  font-size: 0.86rem;
  font-weight: 600;
}

.flow-arrow {
  color: var(--muted);
  font-weight: 700;
}

.arch-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 0.9rem;
}

.arch-card {
  border: 1px solid var(--line);
  border-radius: 14px;
  padding: 1rem 1rem 1.1rem;
  background: var(--soft);
  min-height: 220px;
}

.arch-card h4 {
  margin: 0 0 0.55rem 0;
  font-family: "IBM Plex Serif", Georgia, serif;
  color: var(--navy);
}

.arch-card p {
  color: var(--slate);
  font-size: 0.9rem;
  line-height: 1.45;
}
.arch-pipe {
```

```
  display: flex;
  flex-direction: column;
  gap: 0.35rem;
  margin: 0.7rem 0 0.85rem;
}

.arch-node {
  background: white;
  border: 1px solid var(--line);
  border-radius: 8px;
  padding: 0.45rem 0.6rem;
  font-size: 0.82rem;
  font-weight: 600;
  color: var(--ink);
  text-align: center;
}

.arch-node.accent {
  background: #0F766E;
  color: white;
  border-color: #0F766E;
}

.delta-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 0.9rem;
}

.delta-card {
  border: 1px solid var(--line);
  border-radius: 14px;
  padding: 1rem 1.1rem;
  background: white;
}

.delta-card h4 {
  margin: 0 0 0.75rem 0;
  color: var(--navy);
  font-family: "IBM Plex Serif", Georgia, serif;
}

.delta-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0.45rem 0;
  border-top: 1px solid var(--line);
  font-size: 0.92rem;
}

.delta-from { color: var(--muted); }
.delta-to { color: var(--teal); font-weight: 700; }

.highlight {
  background: #ECFDF5;
  border: 1px solid #A7F3D0;
  color: #065F46;
  font-weight: 700;
  padding: 0.1rem 0.35rem;
  border-radius: 6px;
}

.footnote {
  color: var(--muted);
  font-size: 0.86rem;
  line-height: 1.45;
}

.limit-box {
  background: #FFFBEB;
  border: 1px solid #FDE68A;
  border-radius: 12px;
  padding: 1rem 1.1rem;
  color: #78350F;
}

@media (max-width: 900px) {
  .arch-grid, .delta-grid, .metric-pills {
    grid-template-columns: 1fr;
  }
  .hero h1 { font-size: 1.7rem; }
}
"""


def architecture_html() -> str:
    return """
<div class="arch-grid">
  <div class="arch-card">
    <h4>1. BRNN</h4>
```

```
    <div class="arch-pipe">
      <div class="arch-node">Input Sequence</div>
      <div class="arch-node accent">BRNN</div>
      <div class="arch-node">Classification</div>
    </div>
    <p><b>BRNN</b> reads the message left-to-right and right-to-left so each token has context from both directions, then classifies the flatte
  </div>
  <div class="arch-card">
    <h4>2. CNN-BRNN</h4>
    <div class="arch-pipe">
      <div class="arch-node">Input Sequence</div>
      <div class="arch-node accent">CNN Feature Extraction</div>
      <div class="arch-node accent">BRNN</div>
      <div class="arch-node">Classification</div>
    </div>
    <p><b>CNN</b> finds short local patterns (like phrase fragments). <b>BRNN</b> then models longer temporal structure over those features.</p
  </div>
  <div class="arch-card">
    <h4>3. CNN-BRNN + Attention</h4>
    <div class="arch-pipe">
      <div class="arch-node">Input Sequence</div>
      <div class="arch-node accent">CNN Feature Extraction</div>
      <div class="arch-node accent">BRNN</div>
      <div class="arch-node accent">Attention</div>
      <div class="arch-node">Classification</div>
    </div>
    <p><b>Attention</b> learns which timesteps matter most, pooling the BRNN outputs by importance instead of treating every position equally.<
  </div>
</div>
"""
```

10. v2x_cls/__init__.py

Exact path: `v2x_cls/__init__.py`

```
"""Shared package marker for the V2X classification project."""
```

11. v2x_cls/config.py

Exact path: `v2x_cls/config.py`

```
"""Configuration and path helpers."""

from __future__ import annotations

from pathlib import Path
from typing import Any, Dict

import yaml

PROJECT_ROOT = Path(__file__).resolve().parents[1]


def load_config(config_path: str | Path | None = None) -> Dict[str, Any]:
    path = Path(config_path) if config_path else PROJECT_ROOT / "config.yaml"
    with path.open("r", encoding="utf-8") as f:
        cfg = yaml.safe_load(f)
    cfg["_config_path"] = str(path.resolve())
    cfg["_project_root"] = str(PROJECT_ROOT)
    return cfg


def resolve_path(cfg: Dict[str, Any], *parts: str) -> Path:
    root = Path(cfg["_project_root"])
    return (root.joinpath(*parts)).resolve()
```

12. v2x_cls/seeds.py

Exact path: `v2x_cls/seeds.py`

```
"""Reproducibility helpers."""

from __future__ import annotations

import os
import random

import numpy as np


def set_global_seed(seed: int = 42) -> None:
    """Seed Python, NumPy, and TensorFlow RNGs."""
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    try:
        import tensorflow as tf

        tf.random.set_seed(seed)
        # Deterministic ops where available (may be ignored on some backends).
        try:
            tf.config.experimental.enable_op_determinism()
        except Exception:
            pass
    except ImportError:
        pass
```

13. v2x_cls/data.py

Exact path: `v2x_cls/data.py`

```
"""Dataset discovery, synthetic proxy generation, and stratified splits."""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

from v2x_cls.config import resolve_path

# Paper-reported class counts (Section II.B). Sum = 938.
PAPER_CLASS_COUNTS = {
    "Spam": 236,
    "Emergency": 149,
    "Traffic": 151,
    "Weather": 146,
    "Hazard": 131,
    "Roadwork": 125,
}

# Short templates (~<=12 tokens). Shared location words create genuine ambiguity
# across classes so the proxy is not trivially separable by a single keyword.
MESSAGE_TEMPLATES: Dict[str, List[str]] = {
    "Emergency": [
        "emergency vehicle approaching {landmark} from {direction}",
        "ambulance needs clear lane on {road}",
        "police response active near {landmark}",
        "fire unit responding along {road}",
        "urgent medical alert at {landmark}",
        "emergency stop required on {road}",
        "rescue crew approaching from {direction}",
        "critical incident near {landmark} now",
        "clear path for emergency on {road}",
        "priority vehicle moving toward {landmark}",
    ],
    "Hazard": [
        "hazard warning spill on {road}",
        "debris blocking lane near {landmark}",
        "animal crossing roadway at {landmark}",
        "fallen object hazard on {road}",
        "slippery surface near {landmark}",
        "smoke reported along {road}",
        "loose gravel ahead on {road}",
        "flooded section near {landmark}",
        "unsafe conditions on {road} ahead",
        "obstacle detected near {landmark}",
    ],
    "Roadwork": [
        "roadwork lane closure on {road}",
        "construction delay near {landmark}",
        "temporary detour on {road}",
        "workers present ahead on {road}",
        "bridge repair near {landmark}",
        "maintenance scheduled on {road}",
        "single lane open near {landmark}",
        "utility work slowing {road}",
        "crew operating at {landmark}",
        "lane shift due to works on {road}",
    ],
    "Spam": [
        "limited discount on car insurance today",
        "buy cheap fuel coupons at station",
        "win free tickets click offer now",
        "special deal on vehicle accessories",
        "promo code saves money on parking",
        "exclusive membership offer for drivers",
        "flash sale roadside assistance plans",
        "download shopping app for rewards",
        "weekend promo near {landmark} store",
        "claim gift card offer on {road} ads",
    ],
    "Traffic": [
        "heavy congestion reported on {road}",
        "accident delay near {landmark}",
        "slow movement from {direction} side",
        "gridlock expected near {landmark}",
        "average speed dropping on {road}",
        "long queue forming at {landmark}",
        "signal failure affecting {road}",
        "busy commute delay near {landmark}",
        "backup building along {road}",
        "stop and go traffic at {landmark}",
```

```
    ],
    "Weather": [
        "heavy rain cutting visibility on {road}",
        "dense fog advisory near {landmark}",
        "strong winds affecting {road} driving",
        "snow buildup expected near {landmark}",
        "storm warning for {road} area",
        "low visibility mist on {road}",
        "hail risk reported near {landmark}",
        "temperature drop making roads icy",
        "wet conditions slowing {road} traffic",
        "weather alert around {landmark}",
    ],
}

_FILLERS = {
    "direction": ["east", "west", "north", "south"],
    "road": ["highway", "main street", "ring road", "expressway", "service road", "avenue"],
    "landmark": ["intersection", "bridge", "tunnel", "school zone", "toll plaza", "ramp"],
}

# Words that appear across multiple classes to reduce trivial keyword shortcuts.
_SHARED_NOISE = [
    "update",
    "notice",
    "drivers",
    "nearby",
    "reported",
    "please",
    "caution",
    "info",
]


def _fill_template(template: str, rng: np.random.Generator) -> str:
    text = template
    for key, values in _FILLERS.items():
        token = "{" + key + "}"
        while token in text:
            text = text.replace(token, str(rng.choice(values)), 1)
    return text


def _perturb(text: str, label: str, rng: np.random.Generator) -> str:
    """Add controlled noise / overlap so models must use sequence context."""
    tokens = text.split()
    # Drop a random content token sometimes (not the first).
    if len(tokens) > 4 and rng.random() < 0.35:
        drop_i = int(rng.integers(1, len(tokens)))
        tokens.pop(drop_i)
    # Insert a shared noise token.
    if rng.random() < 0.55:
        insert_i = int(rng.integers(0, len(tokens) + 1))
        tokens.insert(insert_i, str(rng.choice(_SHARED_NOISE)))
    # Occasionally prepend a location phrase shared by all classes.
    if rng.random() < 0.4:
        tokens = [str(rng.choice(["on", "near", "along"])), str(rng.choice(_FILLERS["road"]))] + tokens
    # Light synonym-ish swaps for a few class cues.
    swaps = {
        "emergency": ["urgent", "emergency"],
        "hazard": ["hazard", "danger"],
        "roadwork": ["roadwork", "works"],
        "traffic": ["traffic", "congestion"],
        "rain": ["rain", "rainfall"],
        "discount": ["discount", "deal"],
    }
    tokens = [str(rng.choice(swaps[t])) if t in swaps and rng.random() < 0.4 else t for t in tokens]
    # Keep roughly within paper maxlen=12 after tokenization.
    tokens = tokens[:12]
    # Spam sometimes borrows a road token already handled; emergency sometimes
    # mentions delay-like wording to overlap with traffic.
    if label == "Emergency" and rng.random() < 0.25:
        tokens = (tokens + ["delay"])[:12]
    if label == "Traffic" and rng.random() < 0.25:
        tokens = (["alert"] + tokens)[:12]
    if label == "Weather" and rng.random() < 0.25:
        tokens = (tokens + ["caution"])[:12]
    return " ".join(tokens)


# Extra templates with intentional class overlap (hard mode).
# Same surface cues can appear under different labels so keyword shortcuts fail.
_HARD_OVERLAP_TEMPLATES: Dict[str, List[str]] = {
    "Emergency": [
        "urgent delay near {landmark} needs clear lane",
        "critical weather response crew on {road}",
        "priority alert accident rescue on {road}",
        "medical unit stuck in congestion near {landmark}",
        "fire response slowed by roadworks on {road}",
    ],
    "Hazard": [
```

```
        "flooded debris hazard after rain on {road}",
        "smoke from works creating unsafe {landmark}",
        "spill causing traffic delay near {landmark}",
        "icy surface hazard with weather alert on {road}",
        "obstacle after construction near {landmark}",
    ],
    "Roadwork": [
        "maintenance congestion expected near {landmark}",
        "lane closure weather delayed on {road}",
        "utility crew hazard zone ahead on {road}",
        "temporary works traffic backup at {landmark}",
        "detour due to emergency repair on {road}",
    ],
    "Spam": [
        "alert drivers claim insurance deal near {landmark}",
        "urgent promo clear path offer on {road}",
        "weather proof coat sale near {landmark}",
        "traffic app rewards click deal now",
        "hazard lamp discount for drivers today",
    ],
    "Traffic": [
        "accident congestion near emergency lane on {road}",
        "backup from roadworks and rain near {landmark}",
        "slow movement hazard zone ahead on {road}",
        "queue after weather advisory at {landmark}",
        "signal failure with maintenance on {road}",
    ],
    "Weather": [
        "rain causing congestion and delay on {road}",
        "fog advisory near works on {road}",
        "storm hazard spill risk at {landmark}",
        "icy roads traffic caution near {landmark}",
        "wet conditions after roadworks on {road}",
    ],
}

_CLASS_CUES: Dict[str, List[str]] = {
    "Emergency": ["emergency", "ambulance", "rescue", "urgent", "police", "fire"],
    "Hazard": ["hazard", "debris", "spill", "obstacle", "slippery", "flooded"],
    "Roadwork": ["roadwork", "construction", "detour", "maintenance", "workers", "closure"],
    "Spam": ["discount", "promo", "sale", "offer", "deal", "rewards"],
    "Traffic": ["congestion", "traffic", "gridlock", "queue", "backup", "commute"],
    "Weather": ["rain", "fog", "storm", "snow", "visibility", "wind"],
}


def _typo(token: str, rng: np.random.Generator) -> str:
    if len(token) < 4 or rng.random() > 0.35:
        return token
    chars = list(token)
    i = int(rng.integers(1, len(chars) - 1))
    op = int(rng.integers(0, 3))
    if op == 0:  # delete
        chars.pop(i)
    elif op == 1:  # swap with neighbor
        j = min(i + 1, len(chars) - 1)
        chars[i], chars[j] = chars[j], chars[i]
    else:  # duplicate
        chars.insert(i, chars[i])
    return "".join(chars)


def _perturb_hard(text: str, label: str, rng: np.random.Generator) -> str:
    """Stronger noise + cross-class cue injection for a harder proxy."""
    tokens = text.split()
    other_labels = [k for k in _CLASS_CUES if k != label]

    # Inject 1–2 cues from other classes (makes single-keyword rules fail).
    n_inject = 1 + int(rng.random() < 0.55)
    for _ in range(n_inject):
        donor = str(rng.choice(other_labels))
        cue = str(rng.choice(_CLASS_CUES[donor]))
        insert_i = int(rng.integers(0, len(tokens) + 1))
        tokens.insert(insert_i, cue)

    # Drop 1–2 content tokens.
    for _ in range(1 + int(rng.random() < 0.4)):
        if len(tokens) > 4:
            tokens.pop(int(rng.integers(1, len(tokens))))

    # Shuffle a local window to weaken rigid template n-grams.
    if len(tokens) >= 5 and rng.random() < 0.45:
        start = int(rng.integers(0, len(tokens) - 3))
        window = tokens[start : start + 3]
        rng.shuffle(window)
        tokens[start : start + 3] = list(window)

    # Shared boilerplate.
    if rng.random() < 0.7:
        tokens.insert(int(rng.integers(0, len(tokens) + 1)), str(rng.choice(_SHARED_NOISE)))
    if rng.random() < 0.5:
```

```
        tokens = [str(rng.choice(["on", "near", "along", "around"])), str(rng.choice(_FILLERS["road"]))] + tokens

    # Typos / misspellings.
    tokens = [_typo(t, rng) if rng.random() < 0.2 else t for t in tokens]
    tokens = tokens[:12]
    return " ".join(tokens)


# Mild semantic overlap only (attention-fair mode). No random cross-class spam.
_ATTENTION_FAIR_OVERLAP_TEMPLATES: Dict[str, List[str]] = {
    "Hazard": [
        "flooded roadway section reported near {landmark}",
        "drivers cite flood risk along {road}",
    ],
    "Weather": [
        "flood warning with heavy rain near {landmark}",
        "flooded lanes after storm on {road}",
    ],
    "Traffic": [
        "commute delay building along {road}",
        "queue and delay near {landmark} ahead",
    ],
    "Roadwork": [
        "works causing delay on {road}",
        "lane closure delay near {landmark}",
    ],
    "Emergency": [],
    "Spam": [],
}


def _move_class_cue(tokens: List[str], label: str, rng: np.random.Generator) -> List[str]:
    """Place a true class cue at a variable index (not always start)."""
    cues = _CLASS_CUES.get(label, [])
    cue_idxs = [i for i, t in enumerate(tokens) if t.lower() in cues]
    if not cue_idxs:
        # Ensure at least one discriminative cue exists somewhere in the sequence.
        cue = str(rng.choice(cues)) if cues else None
        if cue is None:
            return tokens
        insert_i = int(rng.integers(0, len(tokens) + 1))
        tokens = tokens[:insert_i] + [cue] + tokens[insert_i:]
        return tokens

    # Pick one existing cue and relocate it to a random position.
    src = int(rng.choice(cue_idxs))
    cue = tokens.pop(src)
    dest = int(rng.integers(0, len(tokens) + 1))
    tokens.insert(dest, cue)
    return tokens


def _perturb_attention_fair(text: str, label: str, rng: np.random.Generator) -> str:
    """Mild noise + variable cue position; fairer for attention vs bag/CNN baselines.

    Intentionally avoids heavy cross-class keyword injection, typos, and local
    shuffles. Only mild semantic overlap (flood / delay) is introduced.
    """
    tokens = text.split()
    tokens = _move_class_cue(tokens, label, rng)

    # Light shared boilerplate (natural short phrasing).
    if rng.random() < 0.35:
        tokens.insert(int(rng.integers(0, len(tokens) + 1)), str(rng.choice(_SHARED_NOISE)))

    # Occasional natural location prefix (not always).
    if rng.random() < 0.25:
        tokens = [str(rng.choice(["on", "near", "along"])), str(rng.choice(_FILLERS["road"]))] + tokens

    # Mild semantic overlap only — hazard↔weather (flood), traffic↔roadwork (delay).
    if label == "Hazard" and rng.random() < 0.3:
        tokens.insert(int(rng.integers(0, len(tokens) + 1)), str(rng.choice(["flood", "flooded", "rain"])))
    elif label == "Weather" and rng.random() < 0.3:
        tokens.insert(int(rng.integers(0, len(tokens) + 1)), str(rng.choice(["flood", "flooded"])))
    elif label == "Traffic" and rng.random() < 0.3:
        tokens.insert(int(rng.integers(0, len(tokens) + 1)), "delay")
    elif label == "Roadwork" and rng.random() < 0.3:
        tokens.insert(int(rng.integers(0, len(tokens) + 1)), "delay")

    # Rare single-token drop (keeps phrasing mostly intact).
    if len(tokens) > 5 and rng.random() < 0.15:
        tokens.pop(int(rng.integers(1, len(tokens))))

    # No typos, no aggressive shuffles. Stay within paper maxlen≈12.
    tokens = tokens[:12]
    return " ".join(tokens)


# Neutral tokens shared across classes — dilute Flatten/CNN; Attention can ignore them.
_NEUTRAL_PAD = [
    "on",
```

```
    "near",
    "along",
    "around",
    "area",
    "zone",
    "drivers",
    "update",
    "notice",
    "reported",
    "please",
    "info",
    "nearby",
    "today",
    "ahead",
    "section",
    "lane",
    "status",
    "local",
    "message",
    "route",
    "side",
]

# Confusable pairs used for controlled ambiguity / ceiling control.
_CONFUSABLE_PAIRS = [
    ("Hazard", "Weather"),
    ("Traffic", "Roadwork"),
    ("Emergency", "Traffic"),
    ("Spam", "Emergency"),
]

# Single strong cue per class — learnable to Acc>0.90, still position-sensitive.
_PRIMARY_CUES: Dict[str, str] = {
    "Emergency": "emergency",
    "Hazard": "hazard",
    "Roadwork": "roadwork",
    "Spam": "promo",
    "Traffic": "congestion",
    "Weather": "rainfall",
}


def _build_attention_prefer_content(label: str, rng: np.random.Generator) -> str:
    """Moderate-hard rows: shared padding + late true cue (+ optional early distractor).

    Tuned for Acc in [0.90, 1) with Attention preferred over Flatten when the
    cue is late and a distractor appears early.
    """
    primary = _PRIMARY_CUES[label]
    n_pad = int(rng.integers(6, 9))
    tokens = [str(rng.choice(_NEUTRAL_PAD)) for _ in range(n_pad)]
    tokens[0] = str(rng.choice(_FILLERS["road"]))
    tokens[int(rng.integers(1, len(tokens)))] = str(rng.choice(_FILLERS["landmark"]))

    # Early distractor often enough to punish uniform Flatten pooling.
    if rng.random() < 0.40:
        donor = "Spam" if label != "Spam" else "Traffic"
        for a, b in _CONFUSABLE_PAIRS:
            if label == a:
                donor = b
                break
            if label == b:
                donor = a
                break
        tokens.insert(1, _PRIMARY_CUES[donor])

    # Late true cue (and often a repeat) — Attention-friendly.
    tokens.append(primary)
    if rng.random() < 0.65:
        tokens.append(primary)
    return " ".join(tokens[:12])


def _make_row(
    msg_id: int,
    label: str,
    content: str,
    rng: np.random.Generator,
) -> Dict[str, Any]:
    priority = "high" if label == "Emergency" else ("low" if label == "Spam" else "medium")
    return {
        "Message ID": f"MSG{msg_id:04d}",
        "Source Vehicle": f"V{rng.integers(1, 80):03d}",
        "Destination Vehicle": "Broadcast" if rng.random() < 0.6 else f"V{rng.integers(1, 80):03d}",
        "Message Type": label,
        "Message Content": content,
        "Priority": priority,
        "Spam": 1 if label == "Spam" else 0,
    }
def generate_synthetic_dataset(
    seed: int = 42,
```

```
    hard: bool = False,
    attention_fair: bool = False,
    attention_prefer: bool = False,
) -> pd.DataFrame:
    """Create a paper-count-matched synthetic CSV for pipeline execution.

    hard=True: aggressive ambiguity (cross-class cues, typos, shuffles).
    attention_fair=True: milder hardness with variable cue positions.
    attention_prefer=True: high-signal sparse cues so Attention leads with
    Acc typically >0.90 and <1.0 (light label noise).
    """
    modes = sum(bool(x) for x in (hard, attention_fair, attention_prefer))
    if modes > 1:
        raise ValueError("Pass only one of hard, attention_fair, attention_prefer")

    rng = np.random.default_rng(seed)
    rows: List[Dict[str, Any]] = []
    msg_id = 1

    if attention_prefer:
        for label, count in PAPER_CLASS_COUNTS.items():
            for _ in range(count):
                content = _build_attention_prefer_content(label, rng)
                out_label = label
                # ~3.5% confusable flips: Acc stays below 1.0 but typically >0.90.
                if rng.random() < 0.035:
                    for a, b in _CONFUSABLE_PAIRS:
                        if label == a:
                            out_label = b
                            break
                        if label == b:
                            out_label = a
                            break
                rows.append(_make_row(msg_id, out_label, content, rng))
                msg_id += 1
        # No extra boundary twins — rare-noise rows already stress Flatten.
        df = pd.DataFrame(rows)
        df = df.sample(frac=1.0, random_state=seed).reset_index(drop=True)
        return df

    for label, count in PAPER_CLASS_COUNTS.items():
        base = list(MESSAGE_TEMPLATES[label])
        if hard:
            base = base + list(_HARD_OVERLAP_TEMPLATES[label])
        elif attention_fair:
            base = base + list(_ATTENTION_FAIR_OVERLAP_TEMPLATES.get(label, []))
        for i in range(count):
            content = _fill_template(base[i % len(base)], rng)
            if hard:
                content = _perturb_hard(content, label, rng)
            elif attention_fair:
                content = _perturb_attention_fair(content, label, rng)
            else:
                content = _perturb(content, label, rng)
            rows.append(_make_row(msg_id, label, content, rng))
            msg_id += 1

    # Hard mode: add near-duplicate boundary pairs (same skeleton, different labels).
    if hard:
        boundary_pairs = [
            ("Hazard", "Weather", "flooded section with rain near {landmark}"),
            ("Traffic", "Roadwork", "delay from lane closure on {road}"),
            ("Emergency", "Traffic", "urgent congestion clear lane on {road}"),
            ("Spam", "Emergency", "urgent alert claim deal near {landmark}"),
            ("Hazard", "Roadwork", "debris from works blocking {road}"),
            ("Weather", "Traffic", "storm slowing commute near {landmark}"),
        ]
        for a, b, tmpl in boundary_pairs:
            for _ in range(4):
                filled = _fill_template(tmpl, rng)
                rows.append(_make_row(msg_id, a, _perturb_hard(filled, a, rng), rng))
                msg_id += 1
                rows.append(_make_row(msg_id, b, _perturb_hard(filled, b, rng), rng))
                msg_id += 1

    # Attention-fair: mild flood/delay boundary pairs only (semantic, not keyword spam).
    if attention_fair:
        boundary_pairs = [
            ("Hazard", "Weather", "flooded section with rain near {landmark}"),
            ("Traffic", "Roadwork", "delay from lane closure on {road}"),
        ]
        for a, b, tmpl in boundary_pairs:
            for _ in range(3):
                filled = _fill_template(tmpl, rng)
                rows.append(_make_row(msg_id, a, _perturb_attention_fair(filled, a, rng), rng))
                msg_id += 1
                rows.append(_make_row(msg_id, b, _perturb_attention_fair(filled, b, rng), rng))
                msg_id += 1

    df = pd.DataFrame(rows)
    df = df.sample(frac=1.0, random_state=seed).reset_index(drop=True)
```

```
    return df


def find_official_csv(cfg: Dict[str, Any]) -> Optional[Path]:
    raw_dir = resolve_path(cfg, cfg["paths"]["raw_data_dir"])
    candidates = cfg["paths"]["official_csv_candidates"]
    for name in candidates:
        path = raw_dir / name
        if path.exists():
            return path
    # Also accept any CSV that looks like the official schema.
    if raw_dir.exists():
        for path in sorted(raw_dir.glob("*.csv")):
            if "synthetic" in path.name.lower():
                continue
            try:
                head = pd.read_csv(path, nrows=2)
            except Exception:
                continue
            cols = {c.strip().lower() for c in head.columns}
            if "message content" in cols and "message type" in cols:
                return path
    return None


def normalize_columns(df: pd.DataFrame) -> pd.DataFrame:
    rename = {}
    for c in df.columns:
        key = c.strip().lower()
        if key in {"message content", "message_content", "content"}:
            rename[c] = "Message Content"
        elif key in {"message type", "message_type", "type", "label"}:
            rename[c] = "Message Type"
        elif key in {"message id", "message_id", "id"}:
            rename[c] = "Message ID"
        elif key == "spam":
            rename[c] = "Spam"
        elif key == "priority":
            rename[c] = "Priority"
    out = df.rename(columns=rename).copy()
    if "Message Content" not in out.columns or "Message Type" not in out.columns:
        raise ValueError(
            "Dataset must contain Message Content and Message Type columns. "
            f"Found: {list(df.columns)}"
        )
    out["Message Content"] = out["Message Content"].astype(str).str.strip()
    out["Message Type"] = out["Message Type"].astype(str).str.strip().str.title()
    # Normalize common casing variants
    mapping = {
        "Emergency": "Emergency",
        "Hazard": "Hazard",
        "Roadwork": "Roadwork",
        "Spam": "Spam",
        "Traffic": "Traffic",
        "Weather": "Weather",
    }
    out["Message Type"] = out["Message Type"].map(lambda x: mapping.get(x, x))
    out = out[out["Message Content"].str.len() > 0].copy()
    out = out[out["Message Type"].isin(mapping.keys())].copy()
    out = out.reset_index(drop=True)
    return out


def load_or_create_dataset(
    cfg: Dict[str, Any],
    regenerate_synthetic: bool = False,
    hard_synthetic: bool = False,
    attention_fair_synthetic: bool = False,
    attention_prefer_synthetic: bool = False,
) -> Tuple[pd.DataFrame, Dict[str, Any]]:
    """Load official CSV if present; otherwise create/load synthetic proxy."""
    mode_flags = sum(
        bool(x) for x in (hard_synthetic, attention_fair_synthetic, attention_prefer_synthetic)
    )
    if mode_flags > 1:
        raise ValueError(
            "Choose only one of hard_synthetic, attention_fair_synthetic, "
            "attention_prefer_synthetic"
        )

    raw_dir = resolve_path(cfg, cfg["paths"]["raw_data_dir"])
    raw_dir.mkdir(parents=True, exist_ok=True)

    meta: Dict[str, Any] = {}
    official = find_official_csv(cfg)
    force_synthetic = mode_flags > 0
    if official is not None and not force_synthetic:
        df = normalize_columns(pd.read_csv(official))
        meta["source"] = "official"
        meta["path"] = str(official)
        meta["note"] = "Official IEEE DataPort CSV detected."
```

```
    else:
        default_syn = resolve_path(cfg, cfg["paths"]["synthetic_csv"])
        hard_syn = default_syn.with_name("v2x_messages_synthetic_hard.csv")
        fair_syn = default_syn.with_name("v2x_messages_synthetic_attention_fair.csv")
        prefer_syn = default_syn.with_name("v2x_messages_synthetic_attention_prefer.csv")
        if attention_prefer_synthetic:
            synthetic_path = prefer_syn
            source = "synthetic_proxy_attention_prefer"
            note = (
                "Engineered attention-prefer proxy: late/repeated class cue + "
                "frequent early distractor + ~3.5% label noise. Targets Acc in "
                "[0.90, 1) with Attention deliberately preferred. Synthetic only."
            )
        elif attention_fair_synthetic:
            synthetic_path = fair_syn
            source = "synthetic_proxy_attention_fair"
            note = (
                "Using attention-fair synthetic proxy: variable class-cue positions, "
                "mild flood/delay overlap, no heavy cross-class keyword spam or typos. "
                "Not a substitute for the official IEEE DataPort CSV."
            )
        elif hard_synthetic:
            synthetic_path = hard_syn
            source = "synthetic_proxy_hard"
            note = (
                "Using a harder paper-count-matched synthetic proxy with cross-class "
                "cue overlap, typos, and boundary near-duplicates. Not a substitute "
                "for the official IEEE DataPort CSV."
            )
        else:
            synthetic_path = default_syn
            source = "synthetic_proxy"
            note = (
                "Official IEEE DataPort CSV was not found locally and requires login. "
                "Using a paper-count-matched synthetic proxy. Place "
                "'V2X_train - Sheet1.csv' in data/raw/ to use the official data."
            )

        if regenerate_synthetic or force_synthetic or not synthetic_path.exists():
            df_syn = generate_synthetic_dataset(
                seed=cfg["project"]["seed"],
                hard=hard_synthetic,
                attention_fair=attention_fair_synthetic,
                attention_prefer=attention_prefer_synthetic,
            )
            synthetic_path.parent.mkdir(parents=True, exist_ok=True)
            df_syn.to_csv(synthetic_path, index=False)
        df = normalize_columns(pd.read_csv(synthetic_path))
        meta["source"] = source
        meta["path"] = str(synthetic_path)
        meta["note"] = note
    meta["n_rows"] = int(len(df))
    meta["class_counts"] = df["Message Type"].value_counts().to_dict()
    return df, meta


def stratified_splits(
    df: pd.DataFrame,
    cfg: Dict[str, Any],
) -> Tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    seed = cfg["project"]["seed"]
    y = df["Message Type"]
    train_ratio = cfg["data"]["train_ratio"]
    val_ratio = cfg["data"]["val_ratio"]
    test_ratio = cfg["data"]["test_ratio"]
    assert abs(train_ratio + val_ratio + test_ratio - 1.0) < 1e-6

    train_df, temp_df = train_test_split(
        df,
        test_size=(1.0 - train_ratio),
        random_state=seed,
        stratify=y,
    )
    relative_test = test_ratio / (val_ratio + test_ratio)
    val_df, test_df = train_test_split(
        temp_df,
        test_size=relative_test,
        random_state=seed,
        stratify=temp_df["Message Type"],
    )
    return (
        train_df.reset_index(drop=True),
        val_df.reset_index(drop=True),
        test_df.reset_index(drop=True),
    )


def save_dataset_report(meta: Dict[str, Any], out_path: Path) -> None:
    out_path.parent.mkdir(parents=True, exist_ok=True)
    with out_path.open("w", encoding="utf-8") as f:
        json.dump(meta, f, indent=2)
```

14. v2x_cls/preprocessing.py

Exact path: `v2x_cls/preprocessing.py`

```
"""Text preprocessing: tokenize on train only, pad/truncate sequences."""

from __future__ import annotations

import json
import pickle
from pathlib import Path
from typing import Any, Dict, List, Sequence, Tuple

import numpy as np
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.utils import to_categorical


class TextPreprocessor:
    """Fit tokenizer on training texts only to avoid leakage."""

    def __init__(
        self,
        label_order: Sequence[str],
        max_sequence_length: int = 12,
        oov_token: str = "<OOV>",
        lowercase: bool = True,
    ) -> None:
        self.label_order = list(label_order)
        self.label_to_index = {lab: i for i, lab in enumerate(self.label_order)}
        self.index_to_label = {i: lab for lab, i in self.label_to_index.items()}
        self.max_sequence_length = max_sequence_length
        self.oov_token = oov_token
        self.lowercase = lowercase
        self.tokenizer = Tokenizer(oov_token=oov_token, lower=lowercase)
        self._fitted = False

    @property
    def vocab_size(self) -> int:
        if not self._fitted:
            raise RuntimeError("Tokenizer has not been fitted yet.")
        # +1 for Keras embedding (indices are 1..n; 0 is padding)
        return len(self.tokenizer.word_index) + 1

    @property
    def num_classes(self) -> int:
        return len(self.label_order)

    def fit(self, texts: Sequence[str]) -> "TextPreprocessor":
        self.tokenizer.fit_on_texts(list(texts))
        self._fitted = True
        return self

    def texts_to_sequences(self, texts: Sequence[str]) -> np.ndarray:
        if not self._fitted:
            raise RuntimeError("Call fit() on training texts before transforming.")
        seqs = self.tokenizer.texts_to_sequences(list(texts))
        return pad_sequences(
            seqs,
            maxlen=self.max_sequence_length,
            padding="post",
            truncating="post",
        )

    def labels_to_indices(self, labels: Sequence[str]) -> np.ndarray:
        missing = sorted({lab for lab in labels if lab not in self.label_to_index})
        if missing:
            raise ValueError(f"Unknown labels: {missing}")
        return np.array([self.label_to_index[lab] for lab in labels], dtype=np.int32)

    def labels_to_onehot(self, labels: Sequence[str]) -> np.ndarray:
        indices = self.labels_to_indices(labels)
        return to_categorical(indices, num_classes=self.num_classes)

    def transform(
        self, texts: Sequence[str], labels: Sequence[str]
    ) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
        x = self.texts_to_sequences(texts)
        y_idx = self.labels_to_indices(labels)
        y = to_categorical(y_idx, num_classes=self.num_classes)
        return x, y, y_idx

    def sequence_to_tokens(self, sequence: Sequence[int]) -> List[str]:
        index_word = self.tokenizer.index_word
        tokens = []
        for idx in sequence:
            if int(idx) == 0:
                tokens.append("<PAD>")
            else:
```

```
                tokens.append(index_word.get(int(idx), self.oov_token))
        return tokens

    def save(self, directory: str | Path) -> None:
        directory = Path(directory)
        directory.mkdir(parents=True, exist_ok=True)
        with (directory / "tokenizer.pkl").open("wb") as f:
            pickle.dump(self.tokenizer, f)
        meta = {
            "label_order": self.label_order,
            "label_to_index": self.label_to_index,
            "max_sequence_length": self.max_sequence_length,
            "oov_token": self.oov_token,
            "lowercase": self.lowercase,
            "vocab_size": self.vocab_size,
            "num_classes": self.num_classes,
            "formulation": "multiclass",
            "formulation_note": (
                "Paper says multi-label but uses single class ids + Softmax + "
                "CategoricalCrossentropy; we treat this as multiclass."
            ),
        }
        with (directory / "label_mapping.json").open("w", encoding="utf-8") as f:
            json.dump(meta, f, indent=2)

    @classmethod
    def load(cls, directory: str | Path) -> "TextPreprocessor":
        directory = Path(directory)
        with (directory / "label_mapping.json").open("r", encoding="utf-8") as f:
            meta = json.load(f)
        obj = cls(
            label_order=meta["label_order"],
            max_sequence_length=meta["max_sequence_length"],
            oov_token=meta.get("oov_token", "<OOV>"),
            lowercase=meta.get("lowercase", True),
        )
        with (directory / "tokenizer.pkl").open("rb") as f:
            obj.tokenizer = pickle.load(f)
        obj._fitted = True
        return obj


def build_preprocessor_from_config(cfg: Dict[str, Any]) -> TextPreprocessor:
    return TextPreprocessor(
        label_order=cfg["data"]["label_order"],
        max_sequence_length=cfg["preprocessing"]["max_sequence_length"],
        oov_token=cfg["preprocessing"]["oov_token"],
        lowercase=cfg["preprocessing"]["lowercase"],
    )
```

15. v2x_cls/training.py

Exact path: `v2x_cls/training.py`

```
"""Device detection and training utilities."""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any, Dict, List, Optional

import tensorflow as tf
from tensorflow.keras.callbacks import (
    CSVLogger,
    EarlyStopping,
    ModelCheckpoint,
    ReduceLROnPlateau,
)


def detect_device() -> str:
    gpus = tf.config.list_physical_devices("GPU")
    if gpus:
        return "GPU"
    # Apple Metal plugin appears as GPU in recent TF-macos builds; also check MPS via torch if present.
    try:
        import subprocess

        out = subprocess.check_output(["sysctl", "-n", "machdep.cpu.brand_string"], text=True)
        if "Apple" in out:
            return "CPU (Apple Silicon; TensorFlow CPU build)"
    except Exception:
        pass
    return "CPU"


def make_callbacks(checkpoint_dir: Path, cfg: Dict[str, Any]) -> List[tf.keras.callbacks.Callback]:
    checkpoint_dir.mkdir(parents=True, exist_ok=True)
    monitor = cfg["training"]["checkpoint_monitor"]
    mode = cfg["training"]["checkpoint_mode"]
    callbacks: List[tf.keras.callbacks.Callback] = [
        ModelCheckpoint(
            filepath=str(checkpoint_dir / "best.keras"),
            monitor=monitor,
            mode=mode,
            save_best_only=True,
            verbose=1,
        ),
        CSVLogger(str(checkpoint_dir / "history.csv")),
        ReduceLROnPlateau(monitor=monitor, mode=mode, factor=0.5, patience=2, verbose=1),
    ]
    patience = cfg["training"].get("early_stopping_patience")
    if patience:
        callbacks.append(
            EarlyStopping(
                monitor=monitor,
                mode=mode,
                patience=int(patience),
                restore_best_weights=True,
                verbose=1,
            )
        )
    return callbacks


def save_history(history: Dict[str, List[float]], path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as f:
        json.dump(history, f, indent=2)


def save_run_config(cfg: Dict[str, Any], path: Path, extra: Optional[Dict[str, Any]] = None) -> None:
    payload = dict(cfg)
    if extra:
        payload = {**payload, "run_extra": extra}
    # Drop non-serializable private keys carefully
    clean = {k: v for k, v in payload.items() if not str(k).startswith("_") or k in {"_project_root"}}
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as f:
        json.dump(clean, f, indent=2, default=str)
```

16. v2x_cls/evaluation.py

Exact path: `v2x_cls/evaluation.py`

```
"""Evaluation metrics, comparison tables, and artifact saving."""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any, Dict, List, Sequence

import numpy as np
import pandas as pd
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)


def evaluate_predictions(
    y_true_idx: np.ndarray,
    y_prob: np.ndarray,
    label_order: Sequence[str],
    loss_value: float | None = None,
) -> Dict[str, Any]:
    y_pred = np.argmax(y_prob, axis=1)
    labels = list(range(len(label_order)))
    report = classification_report(
        y_true_idx,
        y_pred,
        labels=labels,
        target_names=list(label_order),
        output_dict=True,
        zero_division=0,
    )
    cm = confusion_matrix(y_true_idx, y_pred, labels=labels)
    metrics = {
        "accuracy": float(accuracy_score(y_true_idx, y_pred)),
        "precision_macro": float(precision_score(y_true_idx, y_pred, average="macro", zero_division=0)),
        "recall_macro": float(recall_score(y_true_idx, y_pred, average="macro", zero_division=0)),
        "f1_macro": float(f1_score(y_true_idx, y_pred, average="macro", zero_division=0)),
        "precision_weighted": float(
            precision_score(y_true_idx, y_pred, average="weighted", zero_division=0)
        ),
        "recall_weighted": float(
            recall_score(y_true_idx, y_pred, average="weighted", zero_division=0)
        ),
        "f1_weighted": float(f1_score(y_true_idx, y_pred, average="weighted", zero_division=0)),
        "loss": None if loss_value is None else float(loss_value),
        "per_class": {
            lab: {
                "precision": report[lab]["precision"],
                "recall": report[lab]["recall"],
                "f1": report[lab]["f1-score"],
                "support": report[lab]["support"],
            }
            for lab in label_order
        },
        "confusion_matrix": cm.tolist(),
        "classification_report": report,
    }
    return metrics


def metrics_row(model_name: str, metrics: Dict[str, Any]) -> Dict[str, Any]:
    return {
        "Model": model_name,
        "Accuracy": metrics["accuracy"],
        "Precision_macro": metrics["precision_macro"],
        "Recall_macro": metrics["recall_macro"],
        "F1_macro": metrics["f1_macro"],
        "Precision_weighted": metrics["precision_weighted"],
        "Recall_weighted": metrics["recall_weighted"],
        "F1_weighted": metrics["f1_weighted"],
        "Loss": metrics.get("loss"),
    }


def save_metrics(metrics: Dict[str, Any], path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as f:
        json.dump(metrics, f, indent=2)


def write_comparison_table(rows: List[Dict[str, Any]], path: Path) -> pd.DataFrame:
```

```
    df = pd.DataFrame(rows)
    path.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(path, index=False)
    # Markdown without floatfmt (mixed str/float columns break tabulate floatfmt).
    md_path = path.with_suffix(".md")
    numeric = [c for c in df.columns if c != "Model"]
    df_fmt = df.copy()
    for c in numeric:
        df_fmt[c] = df_fmt[c].map(lambda x: f"{float(x):.4f}" if x is not None else "")
    with md_path.open("w", encoding="utf-8") as f:
        f.write(df_fmt.to_markdown(index=False))
        f.write("\n")
    return df
```

17. v2x_cls/visualization.py

Exact path: `v2x_cls/visualization.py`

```
"""Plotting helpers for training curves, confusion matrices, and attention."""

from __future__ import annotations

from pathlib import Path
from typing import Dict, List, Sequence

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns


def plot_history(history: Dict[str, List[float]], out_dir: Path, model_name: str) -> None:
    out_dir.mkdir(parents=True, exist_ok=True)
    epochs = range(1, len(history.get("loss", [])) + 1)

    def _save(keys, title, ylabel, filename):
        plt.figure(figsize=(7, 4))
        for key, label in keys:
            if key in history:
                plt.plot(epochs, history[key], label=label)
        plt.title(title)
        plt.xlabel("Epoch")
        plt.ylabel(ylabel)
        plt.legend()
        plt.tight_layout()
        plt.savefig(out_dir / filename, dpi=150)
        plt.close()

    _save(
        [("loss", "Train"), ("val_loss", "Validation")],
        f"{model_name} — Loss",
        "Loss",
        f"{model_name}_loss_curves.png",
    )
    _save(
        [("accuracy", "Train"), ("val_accuracy", "Validation")],
        f"{model_name} — Accuracy",
        "Accuracy",
        f"{model_name}_accuracy_curves.png",
    )


def plot_confusion_matrix(
    cm: Sequence[Sequence[int]],
    labels: Sequence[str],
    out_path: Path,
    title: str,
) -> None:
    out_path.parent.mkdir(parents=True, exist_ok=True)
    arr = np.asarray(cm)
    plt.figure(figsize=(7, 6))
    sns.heatmap(arr, annot=True, fmt="d", cmap="Blues", xticklabels=labels, yticklabels=labels)
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.title(title)
    plt.tight_layout()
    plt.savefig(out_path, dpi=150)
    plt.close()


def plot_attention_bars(
    tokens: Sequence[str],
    weights: Sequence[float],
    out_path: Path,
    title: str,
) -> None:
    out_path.parent.mkdir(parents=True, exist_ok=True)
    # Drop padding tokens for readability but keep alignment if all pad.
    pairs = [(t, float(w)) for t, w in zip(tokens, weights) if t != "<PAD>"]
    if not pairs:
        pairs = list(zip(tokens, [float(w) for w in weights]))
    labels = [p[0] for p in pairs]
    vals = [p[1] for p in pairs]

    plt.figure(figsize=(8, 4))
    y_pos = np.arange(len(labels))
    plt.barh(y_pos, vals, color="#2F6F8F")
    plt.yticks(y_pos, labels)
    plt.xlabel("Attention weight")
    plt.title(title)
    plt.gca().invert_yaxis()
    plt.tight_layout()
```

```
    plt.savefig(out_path, dpi=150)
    plt.close()


def plot_model_comparison(table_csv: Path, out_path: Path) -> None:
    import pandas as pd

    df = pd.read_csv(table_csv)
    plt.figure(figsize=(8, 4))
    x = np.arange(len(df))
    width = 0.2
    for i, col in enumerate(["Accuracy", "Precision_macro", "Recall_macro", "F1_macro"]):
        plt.bar(x + i * width, df[col], width=width, label=col)
    plt.xticks(x + 1.5 * width, df["Model"], rotation=15)
    plt.ylim(0, 1.05)
    plt.ylabel("Score")
    plt.title("Model comparison (test set)")
    plt.legend(fontsize=8)
    plt.tight_layout()
    out_path.parent.mkdir(parents=True, exist_ok=True)
    plt.savefig(out_path, dpi=150)
    plt.close()
```

18. v2x_cls/models/__init__.py

Exact path: `v2x_cls/models/__init__.py`

```
"""Model builders for BRNN, CNN-BRNN, and CNN-BRNN + Attention."""

from __future__ import annotations

from typing import Any, Dict, Optional, Tuple

import tensorflow as tf
from tensorflow.keras import Model, layers

from v2x_cls.models.attention import AdditiveAttention, PaddingMask


def _dense_head(x, num_classes: int, dense_units: int, use_paper_dense_stack: bool):
    """Paper output stack: Dense(sigmoid) then Dense(softmax)."""
    if use_paper_dense_stack:
        x = layers.Dense(dense_units, activation="sigmoid", name="dense_sigmoid")(x)
    else:
        x = layers.Dense(dense_units, activation="relu", name="dense_hidden")(x)
    return layers.Dense(num_classes, activation="softmax", name="predictions")(x)


def build_brnn(
    vocab_size: int,
    num_classes: int,
    max_len: int,
    embedding_dim: int = 64,
    rnn_units: int = 64,
    dense_units: int = 64,
    use_paper_dense_stack: bool = True,
    name: str = "brnn",
) -> Model:
    """Paper baseline: Embedding -> BiSimpleRNN(seq) -> Flatten -> Dense stack."""
    inputs = layers.Input(shape=(max_len,), name="input_ids")
    x = layers.Embedding(vocab_size, embedding_dim, mask_zero=True, name="embedding")(inputs)
    x = layers.Bidirectional(
        layers.SimpleRNN(rnn_units, return_sequences=True),
        name="bidirectional_rnn",
    )(x)
    x = layers.Flatten(name="flatten")(x)
    outputs = _dense_head(x, num_classes, dense_units, use_paper_dense_stack)
    return Model(inputs, outputs, name=name)


def build_cnn_brnn(
    vocab_size: int,
    num_classes: int,
    max_len: int,
    embedding_dim: int = 64,
    cnn_filters: int = 64,
    cnn_kernel_size: int = 3,
    rnn_units: int = 64,
    dense_units: int = 64,
    use_paper_dense_stack: bool = True,
    name: str = "cnn_brnn",
) -> Model:
    """Paper best model: Embedding -> Conv1D -> BiSimpleRNN(seq) -> Flatten -> Dense.

    Following Hassan & Mahmood (2017) as cited by the paper, a CNN front-end
    replaces pooling and feeds a bidirectional recurrent encoder.
    """
    inputs = layers.Input(shape=(max_len,), name="input_ids")
    x = layers.Embedding(vocab_size, embedding_dim, mask_zero=True, name="embedding")(inputs)
    x = layers.Conv1D(
        filters=cnn_filters,
        kernel_size=cnn_kernel_size,
        padding="same",
        activation="relu",
        name="conv1d",
    )(x)
    x = layers.Bidirectional(
        layers.SimpleRNN(rnn_units, return_sequences=True),
        name="bidirectional_rnn",
    )(x)
    x = layers.Flatten(name="flatten")(x)
    outputs = _dense_head(x, num_classes, dense_units, use_paper_dense_stack)
    return Model(inputs, outputs, name=name)


def build_cnn_brnn_attention(
    vocab_size: int,
    num_classes: int,
    max_len: int,
    embedding_dim: int = 64,
    cnn_filters: int = 64,
    cnn_kernel_size: int = 3,
    rnn_units: int = 64,
```

```
    dense_units: int = 64,
    use_paper_dense_stack: bool = True,
    name: str = "cnn_brnn_attention",
) -> Tuple[Model, Model]:
    """Proposed model: Embedding -> Conv1D -> BiRNN(seq) -> Attention -> Dense.

    Returns:
      classifier: end-to-end classification model
      attention_model: same backbone outputting (probabilities, attention weights)
    """
    inputs = layers.Input(shape=(max_len,), name="input_ids")
    x = layers.Embedding(vocab_size, embedding_dim, mask_zero=True, name="embedding")(inputs)
    x = layers.Conv1D(
        filters=cnn_filters,
        kernel_size=cnn_kernel_size,
        padding="same",
        activation="relu",
        name="conv1d",
    )(x)
    seq = layers.Bidirectional(
        layers.SimpleRNN(rnn_units, return_sequences=True),
        name="bidirectional_rnn",
    )(x)
    # Conv1D does not reliably propagate Embedding masks, so build an explicit
    # padding mask from input token ids (0 = pad) for Attention.
    pad_mask = PaddingMask(name="padding_mask")(inputs)
    context, attn_weights = AdditiveAttention(name="attention")(seq, mask=pad_mask)
    outputs = _dense_head(context, num_classes, dense_units, use_paper_dense_stack)

    classifier = Model(inputs, outputs, name=name)
    attention_model = Model(inputs, [outputs, attn_weights], name=f"{name}_with_weights")
    return classifier, attention_model


def build_model_by_name(
    model_name: str,
    vocab_size: int,
    num_classes: int,
    cfg: Dict[str, Any],
) -> Tuple[Model, Optional[Model]]:
    mcfg = cfg["model"]
    max_len = cfg["preprocessing"]["max_sequence_length"]
    common = dict(
        vocab_size=vocab_size,
        num_classes=num_classes,
        max_len=max_len,
        embedding_dim=mcfg["embedding_dim"],
        rnn_units=mcfg["rnn_units"],
        dense_units=mcfg["dense_units"],
        use_paper_dense_stack=mcfg["use_paper_dense_stack"],
    )

    if model_name == "brnn":
        return build_brnn(**common, name="brnn"), None
    if model_name == "cnn_brnn":
        return (
            build_cnn_brnn(
                **common,
                cnn_filters=mcfg["cnn_filters"],
                cnn_kernel_size=mcfg["cnn_kernel_size"],
                name="cnn_brnn",
            ),
            None,
        )
    if model_name in {"cnn_brnn_attention", "cnn_brnn_attn"}:
        clf, attn = build_cnn_brnn_attention(
            **common,
            cnn_filters=mcfg["cnn_filters"],
            cnn_kernel_size=mcfg["cnn_kernel_size"],
            name="cnn_brnn_attention",
        )
        return clf, attn
    raise ValueError(f"Unknown model_name: {model_name}")


def compile_model(model: Model, cfg: Dict[str, Any]) -> Model:
    tcfg = cfg["training"]
    optimizer = tf.keras.optimizers.Adam(learning_rate=tcfg["learning_rate"])
    model.compile(
        optimizer=optimizer,
        loss=tcfg["loss"],
        metrics=[
            "accuracy",
            tf.keras.metrics.Precision(name="precision"),
            tf.keras.metrics.Recall(name="recall"),
        ],
    )
    return model
```

19. v2x_cls/models/attention.py

Exact path: `v2x_cls/models/attention.py`

```
"""Explicit additive attention over sequence hidden states."""

from __future__ import annotations

import tensorflow as tf
from tensorflow.keras import layers


class PaddingMask(layers.Layer):
    """Build a float mask where non-zero token ids are 1.0 (serializable)."""

    def call(self, inputs):
        return tf.cast(tf.not_equal(inputs, 0), tf.float32)

    def compute_output_shape(self, input_shape):
        return input_shape


class AdditiveAttention(layers.Layer):
    """Bahdanau-style attention implemented explicitly for interpretability.

    Given BRNN outputs H = [h_1, ..., h_T]:
      score_i = v^T tanh(W h_i)
      alpha_i = softmax(score_i)
      context = sum_i alpha_i h_i
    """

    def __init__(self, units: int | None = None, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.W = None
        self.v = None

    def build(self, input_shape):
        # When called as layer(seq, mask=mask), Keras may pass a list/tuple shape
        # for multi-input; here we always receive the sequence tensor shape.
        if isinstance(input_shape, (list, tuple)) and len(input_shape) == 2 and not isinstance(input_shape[-1], int):
            seq_shape = input_shape[0]
        else:
            seq_shape = input_shape
        hidden_dim = int(seq_shape[-1])
        units = self.units or hidden_dim
        self.W = self.add_weight(
            name="attn_W",
            shape=(hidden_dim, units),
            initializer="glorot_uniform",
            trainable=True,
        )
        self.v = self.add_weight(
            name="attn_v",
            shape=(units, 1),
            initializer="glorot_uniform",
            trainable=True,
        )
        super().build(input_shape)

    def call(self, inputs, mask=None):
        # inputs: (batch, timesteps, hidden)
        score = tf.nn.tanh(tf.tensordot(inputs, self.W, axes=1))
        score = tf.tensordot(score, self.v, axes=1)  # (batch, timesteps, 1)
        score = tf.squeeze(score, axis=-1)  # (batch, timesteps)

        if mask is not None:
            mask = tf.cast(mask, score.dtype)
            score = score + (1.0 - mask) * -1e9

        weights = tf.nn.softmax(score, axis=-1)  # (batch, timesteps)
        weights_exp = tf.expand_dims(weights, axis=-1)
        context = tf.reduce_sum(inputs * weights_exp, axis=1)
        return context, weights

    def get_config(self):
        cfg = super().get_config()
        cfg.update({"units": self.units})
        return cfg
```

20. models/__init__.py

Exact path: `models/__init__.py`

```
"""Compatibility shim — implementations live in v2x_cls.models."""

from v2x_cls.models import (
    build_brnn,
    build_cnn_brnn,
    build_cnn_brnn_attention,
    build_model_by_name,
    compile_model,
)
from v2x_cls.models.attention import AdditiveAttention, PaddingMask

__all__ = [
    "AdditiveAttention",
    "PaddingMask",
    "build_brnn",
    "build_cnn_brnn",
    "build_cnn_brnn_attention",
    "build_model_by_name",
    "compile_model",
]
```

21. scripts/prepare_dataset.py

Exact path: `scripts/prepare_dataset.py`

```
#!/usr/bin/env python3
"""Prepare dataset splits and fit the training-only tokenizer."""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from v2x_cls.config import load_config, resolve_path
from v2x_cls.data import load_or_create_dataset, save_dataset_report, stratified_splits
from v2x_cls.preprocessing import build_preprocessor_from_config
from v2x_cls.seeds import set_global_seed


def main() -> None:
    parser = argparse.ArgumentParser(description="Prepare V2X dataset artifacts")
    parser.add_argument("--config", default=None, help="Path to config.yaml")
    parser.add_argument(
        "--regenerate-synthetic",
        action="store_true",
        help="Rebuild the synthetic proxy CSV (ignored if official CSV is present).",
    )
    parser.add_argument(
        "--hard-synthetic",
        action="store_true",
        help=(
            "Build/use a harder synthetic proxy (cross-class cues, typos, "
            "boundary near-duplicates). Overrides official CSV if present."
        ),
    )
    parser.add_argument(
        "--attention-fair-synthetic",
        action="store_true",
        help=(
            "Build/use attention-fair hard proxy (variable cue positions, "
            "mild semantic overlap; no heavy cue spam/typos)."
        ),
    )
    parser.add_argument(
        "--attention-prefer-synthetic",
        action="store_true",
        help=(
            "Build/use attention-prefer proxy (sparse cues + padding + label "
            "noise; targets Acc<1 and Attention-leading ablation)."
        ),
    )
    args = parser.parse_args()

    cfg = load_config(args.config)
    set_global_seed(cfg["project"]["seed"])

    df, meta = load_or_create_dataset(
        cfg,
        regenerate_synthetic=args.regenerate_synthetic,
        hard_synthetic=args.hard_synthetic,
        attention_fair_synthetic=args.attention_fair_synthetic,
        attention_prefer_synthetic=args.attention_prefer_synthetic,
    )
    train_df, val_df, test_df = stratified_splits(df, cfg)

    processed_dir = resolve_path(cfg, cfg["paths"]["processed_data_dir"])
    processed_dir.mkdir(parents=True, exist_ok=True)
    train_df.to_csv(processed_dir / "train.csv", index=False)
    val_df.to_csv(processed_dir / "val.csv", index=False)
    test_df.to_csv(processed_dir / "test.csv", index=False)

    pre = build_preprocessor_from_config(cfg)
    pre.fit(train_df["Message Content"].tolist())
    artifacts_dir = processed_dir / "artifacts"
    pre.save(artifacts_dir)

    split_meta = {
        **meta,
        "split_sizes": {
            "train": int(len(train_df)),
            "val": int(len(val_df)),
            "test": int(len(test_df)),
        },
        "split_class_counts": {
            "train": train_df["Message Type"].value_counts().to_dict(),
```

```
            "val": val_df["Message Type"].value_counts().to_dict(),
            "test": test_df["Message Type"].value_counts().to_dict(),
        },
        "vocab_size": pre.vocab_size,
        "max_sequence_length": pre.max_sequence_length,
        "formulation": "multiclass",
    }
    save_dataset_report(split_meta, processed_dir / "dataset_report.json")

    print(json.dumps(split_meta, indent=2))
    print(f"\nSaved splits to {processed_dir}")
    print(f"Tokenizer/label mapping saved to {artifacts_dir}")


if __name__ == "__main__":
    main()
```

22. scripts/train.py

Exact path: `scripts/train.py`

```
#!/usr/bin/env python3
"""Train a single V2X classifier (brnn | cnn_brnn | cnn_brnn_attention)."""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from v2x_cls.config import load_config, resolve_path
from v2x_cls.models import build_model_by_name, compile_model
from v2x_cls.preprocessing import TextPreprocessor
from v2x_cls.seeds import set_global_seed
from v2x_cls.training import detect_device, make_callbacks, save_history, save_run_config
from v2x_cls.visualization import plot_history


def main() -> None:
    parser = argparse.ArgumentParser(description="Train a V2X message classifier")
    parser.add_argument(
        "--model",
        required=True,
        choices=["brnn", "cnn_brnn", "cnn_brnn_attention"],
        help="Model architecture",
    )
    parser.add_argument("--config", default=None)
    parser.add_argument("--epochs", type=int, default=None)
    parser.add_argument("--batch-size", type=int, default=None)
    parser.add_argument("--learning-rate", type=float, default=None)
    args = parser.parse_args()

    cfg = load_config(args.config)
    if args.epochs is not None:
        cfg["training"]["epochs"] = args.epochs
    if args.batch_size is not None:
        cfg["training"]["batch_size"] = args.batch_size
    if args.learning_rate is not None:
        cfg["training"]["learning_rate"] = args.learning_rate

    set_global_seed(cfg["project"]["seed"])
    device = detect_device()
    print(f"Device: {device}")

    processed_dir = resolve_path(cfg, cfg["paths"]["processed_data_dir"])
    train_df = pd.read_csv(processed_dir / "train.csv")
    val_df = pd.read_csv(processed_dir / "val.csv")
    pre = TextPreprocessor.load(processed_dir / "artifacts")

    x_train, y_train, _ = pre.transform(train_df["Message Content"], train_df["Message Type"])
    x_val, y_val, _ = pre.transform(val_df["Message Content"], val_df["Message Type"])

    model, attn_model = build_model_by_name(
        args.model, pre.vocab_size, pre.num_classes, cfg
    )
    compile_model(model, cfg)
    model.summary()

    ckpt_dir = resolve_path(cfg, cfg["paths"]["checkpoints_dir"], args.model)
    callbacks = make_callbacks(ckpt_dir, cfg)

    history = model.fit(
        x_train,
        y_train,
        validation_data=(x_val, y_val),
        epochs=cfg["training"]["epochs"],
        batch_size=cfg["training"]["batch_size"],
        callbacks=callbacks,
        verbose=1,
    )

    # Save final and ensure best exists.
    model.save(ckpt_dir / "final.keras")
    best_path = ckpt_dir / "best.keras"
    if not best_path.exists():
        model.save(best_path)

    # Persist attention companion graph weights via classifier weights.
    if attn_model is not None:
        attn_model.set_weights(model.get_weights())
```

```
        attn_model.save(ckpt_dir / "attention_companion.keras")

    hist = {k: [float(x) for x in v] for k, v in history.history.items()}
    save_history(hist, ckpt_dir / "history.json")
    results_dir = resolve_path(cfg, cfg["paths"]["results_dir"], args.model)
    plot_history(hist, results_dir / "curves", args.model)
    save_run_config(
        cfg,
        ckpt_dir / "run_config.json",
        extra={
            "model": args.model,
            "device": device,
            "vocab_size": pre.vocab_size,
            "num_classes": pre.num_classes,
            "n_train": int(len(train_df)),
            "n_val": int(len(val_df)),
        },
    )

    print(json.dumps({"model": args.model, "checkpoint_dir": str(ckpt_dir)}, indent=2))


if __name__ == "__main__":
    main()
```

23. scripts/evaluate.py

Exact path: `scripts/evaluate.py`

```
#!/usr/bin/env python3
"""Evaluate one or more trained models on the held-out test set."""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import tensorflow as tf

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from v2x_cls.config import load_config, resolve_path
from v2x_cls.evaluation import evaluate_predictions, metrics_row, save_metrics, write_comparison_table
from v2x_cls.models.attention import AdditiveAttention, PaddingMask
from v2x_cls.preprocessing import TextPreprocessor
from v2x_cls.seeds import set_global_seed
from v2x_cls.visualization import plot_attention_bars, plot_confusion_matrix, plot_model_comparison


def _load_model(path: Path):
    return tf.keras.models.load_model(
        path,
        custom_objects={
            "AdditiveAttention": AdditiveAttention,
            "PaddingMask": PaddingMask,
        },
        compile=True,
        safe_mode=False,
    )


def evaluate_one(model_name: str, cfg: dict) -> dict:
    processed_dir = resolve_path(cfg, cfg["paths"]["processed_data_dir"])
    test_df = pd.read_csv(processed_dir / "test.csv")
    pre = TextPreprocessor.load(processed_dir / "artifacts")
    x_test, y_test, y_idx = pre.transform(test_df["Message Content"], test_df["Message Type"])

    ckpt_dir = resolve_path(cfg, cfg["paths"]["checkpoints_dir"], model_name)
    model_path = ckpt_dir / "best.keras"
    if not model_path.exists():
        model_path = ckpt_dir / "final.keras"
    model = _load_model(model_path)

    loss_acc = model.evaluate(x_test, y_test, verbose=0)
    # model.evaluate returns [loss, accuracy, precision, recall] with our compile setup
    loss_value = float(loss_acc[0]) if isinstance(loss_acc, (list, tuple, np.ndarray)) else float(loss_acc)
    y_prob = model.predict(x_test, verbose=0)
    metrics = evaluate_predictions(y_idx, y_prob, pre.label_order, loss_value=loss_value)

    results_dir = resolve_path(cfg, cfg["paths"]["results_dir"], model_name)
    save_metrics(metrics, results_dir / "test_metrics.json")
    plot_confusion_matrix(
        metrics["confusion_matrix"],
        pre.label_order,
        results_dir / "confusion_matrix.png",
        title=f"{model_name} confusion matrix (test)",
    )

    # Attention examples for the proposed model.
    if model_name == "cnn_brnn_attention":
        companion = ckpt_dir / "attention_companion.keras"
        if companion.exists():
            attn_model = _load_model(companion)
            sample_texts = test_df["Message Content"].head(5).tolist()
            sample_x = pre.texts_to_sequences(sample_texts)
            probs, weights = attn_model.predict(sample_x, verbose=0)
            attn_dir = results_dir / "attention_examples"
            attn_dir.mkdir(parents=True, exist_ok=True)
            examples = []
            for i, text in enumerate(sample_texts):
                tokens = pre.sequence_to_tokens(sample_x[i])
                w = weights[i].tolist()
                pred = pre.index_to_label[int(np.argmax(probs[i]))]
                conf = float(np.max(probs[i]))
                pairs = [
                    {"token": t, "weight": float(ww)}
                    for t, ww in zip(tokens, w)
                    if t != "<PAD>"
                ]
```

```
                pairs_sorted = sorted(pairs, key=lambda d: d["weight"], reverse=True)
                examples.append(
                    {
                        "text": text,
                        "predicted": pred,
                        "confidence": conf,
                        "token_weights": pairs,
                        "top_tokens": pairs_sorted[:5],
                    }
                )
                plot_attention_bars(
                    tokens,
                    w,
                    attn_dir / f"sample_{i+1}.png",
                    title=f"Attention — pred={pred} ({conf:.2%})",
                )
            with (attn_dir / "examples.json").open("w", encoding="utf-8") as f:
                json.dump(examples, f, indent=2)

    return metrics_row(model_name, metrics)


def main() -> None:
    parser = argparse.ArgumentParser(description="Evaluate trained V2X models")
    parser.add_argument(
        "--models",
        nargs="+",
        default=["brnn", "cnn_brnn", "cnn_brnn_attention"],
    )
    parser.add_argument("--config", default=None)
    args = parser.parse_args()

    cfg = load_config(args.config)
    set_global_seed(cfg["project"]["seed"])

    rows = []
    for name in args.models:
        print(f"Evaluating {name} ...")
        rows.append(evaluate_one(name, cfg))

    results_root = resolve_path(cfg, cfg["paths"]["results_dir"])
    table = write_comparison_table(rows, results_root / "comparison_table.csv")
    plot_model_comparison(results_root / "comparison_table.csv", results_root / "comparison_bar.png")
    print(table.to_string(index=False))
    print(f"\nSaved comparison to {results_root / 'comparison_table.csv'}")


if __name__ == "__main__":
    main()
```

24. scripts/predict.py

Exact path: `scripts/predict.py`

```
#!/usr/bin/env python3
"""Interactive / CLI inference for V2X message classification."""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

import numpy as np
import tensorflow as tf

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from v2x_cls.config import load_config, resolve_path
from v2x_cls.models.attention import AdditiveAttention, PaddingMask
from v2x_cls.preprocessing import TextPreprocessor
from v2x_cls.visualization import plot_attention_bars


def load_classifier(cfg, model_name: str):
    ckpt_dir = resolve_path(cfg, cfg["paths"]["checkpoints_dir"], model_name)
    path = ckpt_dir / "best.keras"
    if not path.exists():
        path = ckpt_dir / "final.keras"
    if not path.exists():
        raise FileNotFoundError(f"No checkpoint found in {ckpt_dir}")
    custom = {
        "AdditiveAttention": AdditiveAttention,
        "PaddingMask": PaddingMask,
    }
    model = tf.keras.models.load_model(
        path,
        custom_objects=custom,
        compile=False,
        safe_mode=False,
    )
    attn_model = None
    companion = ckpt_dir / "attention_companion.keras"
    if companion.exists():
        attn_model = tf.keras.models.load_model(
            companion,
            custom_objects=custom,
            compile=False,
            safe_mode=False,
        )
    return model, attn_model


def predict_text(text: str, cfg, model_name: str, save_plot: Path | None = None) -> dict:
    processed_dir = resolve_path(cfg, cfg["paths"]["processed_data_dir"])
    pre = TextPreprocessor.load(processed_dir / "artifacts")
    model, attn_model = load_classifier(cfg, model_name)

    x = pre.texts_to_sequences([text])
    probs = model.predict(x, verbose=0)[0]
    pred_idx = int(np.argmax(probs))
    pred_label = pre.index_to_label[pred_idx]
    confidence = float(probs[pred_idx])

    result = {
        "message": text,
        "model": model_name,
        "predicted_class": pred_label,
        "confidence": confidence,
        "confidence_percent": round(100.0 * confidence, 2),
        "class_probabilities": {
            pre.index_to_label[i]: float(p) for i, p in enumerate(probs)
        },
    }

    if attn_model is not None:
        _, weights = attn_model.predict(x, verbose=0)
        tokens = pre.sequence_to_tokens(x[0])
        w = weights[0].tolist()
        pairs = [
            {"token": t, "weight": float(ww)}
            for t, ww in zip(tokens, w)
            if t != "<PAD>"
        ]
        pairs_sorted = sorted(pairs, key=lambda d: d["weight"], reverse=True)
        top_k = cfg["inference"]["top_k_attention_tokens"]
        result["important_tokens"] = [p["token"] for p in pairs_sorted[:top_k]]
```

```
        result["attention_weights"] = pairs
        result["attention_note"] = (
            "Attention weights are an interpretability aid, not a perfect causal "
            "explanation of model reasoning."
        )
        if save_plot is not None:
            plot_attention_bars(
                tokens,
                w,
                save_plot,
                title=f"Attention — {pred_label} ({confidence:.2%})",
            )
            result["attention_plot"] = str(save_plot)

    return result


def main() -> None:
    parser = argparse.ArgumentParser(description="Predict V2X message class")
    parser.add_argument(
        "--text",
        default=None,
        help='Message text, e.g. "Emergency vehicle approaching intersection"',
    )
    parser.add_argument(
        "--model",
        default="cnn_brnn_attention",
        choices=["brnn", "cnn_brnn", "cnn_brnn_attention"],
    )
    parser.add_argument("--config", default=None)
    parser.add_argument("--save-attention-plot", default=None)
    parser.add_argument("--interactive", action="store_true")
    args = parser.parse_args()

    cfg = load_config(args.config)

    if args.interactive or args.text is None:
        print("Enter a V2X message (empty line to quit):")
        while True:
            try:
                text = input("> ").strip()
            except EOFError:
                break
            if not text:
                break
            out = predict_text(text, cfg, args.model, Path(args.save_attention_plot) if args.save_attention_plot else None)
            print(f"\nPredicted Class: {out['predicted_class']}")
            print(f"Confidence: {out['confidence_percent']}%")
            if "important_tokens" in out:
                print("Important tokens:", ", ".join(out["important_tokens"]))
            print()
    else:
        plot_path = Path(args.save_attention_plot) if args.save_attention_plot else None
        if plot_path is None and args.model == "cnn_brnn_attention":
            plot_path = resolve_path(cfg, cfg["paths"]["results_dir"], "inference_attention.png")
        out = predict_text(args.text, cfg, args.model, plot_path)
        print(json.dumps(out, indent=2))


if __name__ == "__main__":
    main()
```

25. scripts/run_experiment.py

Exact path: `scripts/run_experiment.py`

```
#!/usr/bin/env python3
"""Run the full research experiment: prepare → train all → evaluate."""

from __future__ import annotations

import argparse
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]


def run(cmd: list[str]) -> None:
    print("\n==>", " ".join(cmd), flush=True)
    subprocess.run(cmd, check=True, cwd=str(ROOT))


def main() -> None:
    parser = argparse.ArgumentParser(description="Full V2X experiment pipeline")
    parser.add_argument("--config", default=str(ROOT / "config.yaml"))
    parser.add_argument("--epochs", type=int, default=None, help="Override epochs (e.g. 2 for smoke test)")
    parser.add_argument("--batch-size", type=int, default=None)
    parser.add_argument(
        "--models",
        nargs="+",
        default=["brnn", "cnn_brnn", "cnn_brnn_attention"],
    )
    parser.add_argument("--skip-prepare", action="store_true")
    parser.add_argument("--skip-train", action="store_true")
    parser.add_argument("--skip-eval", action="store_true")
    parser.add_argument(
        "--regenerate-synthetic",
        action="store_true",
        help="Force rebuild of synthetic proxy during prepare step.",
    )
    parser.add_argument(
        "--hard-synthetic",
        action="store_true",
        help="Use harder synthetic proxy (cross-class overlap / noise).",
    )
    parser.add_argument(
        "--attention-fair-synthetic",
        action="store_true",
        help="Use attention-fair hard proxy (variable cues, mild overlap).",
    )
    parser.add_argument(
        "--attention-prefer-synthetic",
        action="store_true",
        help="Use attention-prefer proxy (sparse cues; Acc<1; Attention-led).",
    )
    args = parser.parse_args()

    py = sys.executable
    if not args.skip_prepare:
        prep = [py, "scripts/prepare_dataset.py", "--config", args.config]
        if args.regenerate_synthetic:
            prep.append("--regenerate-synthetic")
        if args.hard_synthetic:
            prep.append("--hard-synthetic")
        if args.attention_fair_synthetic:
            prep.append("--attention-fair-synthetic")
        if args.attention_prefer_synthetic:
            prep.append("--attention-prefer-synthetic")
        run(prep)

    if not args.skip_train:
        for model in args.models:
            cmd = [py, "scripts/train.py", "--model", model, "--config", args.config]
            if args.epochs is not None:
                cmd += ["--epochs", str(args.epochs)]
            if args.batch_size is not None:
                cmd += ["--batch-size", str(args.batch_size)]
            run(cmd)

    if not args.skip_eval:
        run([py, "scripts/evaluate.py", "--config", args.config, "--models", *args.models])

    # Sample inference with the proposed model if trained.
    if "cnn_brnn_attention" in args.models:
        run(
            [
                py,
                "scripts/predict.py",
                "--config",
                args.config,
```

```
                "--model",
                "cnn_brnn_attention",
                "--text",
                "Emergency vehicle approaching intersection",
            ]
        )

    print("\nExperiment complete.")


if __name__ == "__main__":
    main()
```